CAREERSPHERE AI: AN EXPLAINABLE, SKILL-AWARE CAMPUS PLACEMENT PLATFORM WITH OPPORTUNITY MATH, SKILL ROADMAPS AND A GROUNDED AI COPILOT

A Project Report

submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

V. VENKATA RUPIN	**2311CS010689**
E. MANIDEEP REDDY	**2311CS010182**
J. PRUTHVI SAI	**2311CS010264**
S. JAFFER ALI	**2311CS010639**

Under the guidance of

[Guide name]

[Designation], Department of Computer Science and Engineering

[College logo]

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

[COLLEGE / UNIVERSITY NAME]

[City, State – PIN]

2026–2027

CERTIFICATE

This is to certify that the project report entitled **"CareerSphere AI: An Explainable, Skill-Aware Campus Placement Platform with Opportunity Math, Skill Roadmaps and a Grounded AI Copilot"** is a bonafide record of the work carried out by **V. Venkata Rupin (2311CS010689), E. Manideep Reddy (2311CS010182), J. Pruthvi Sai (2311CS010264) and S. Jaffer Ali (2311CS010639)** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at College / University name during the academic year 2026–2027. The results in this report have not been submitted to any other university or institute for the award of any degree or diploma.

Guide name
Project Guide
Designation, Dept. of CSE

Head of Department name
Head of the Department
Dept. of CSE

External Examiner

Date of viva-voce: _____

DECLARATION

We hereby declare that the project report entitled **"CareerSphere AI: An Explainable, Skill-Aware Campus Placement Platform with Opportunity Math, Skill Roadmaps and a Grounded AI Copilot"**, submitted to the Department of Computer Science and Engineering, College / University name, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is a record of original work done by us under the guidance of Guide name, Designation. The work has not been submitted, in part or in full, to any other university or institute for the award of any degree or diploma. All data in this report is fictional demo data created for the project.

V. Venkata Rupin (2311CS010689) Signature: _____
E. Manideep Reddy (2311CS010182) Signature: _____
J. Pruthvi Sai (2311CS010264) Signature: _____
S. Jaffer Ali (2311CS010639) Signature: _____

Place: City
Date: _____

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Guide name, Designation, Department of Computer Science and Engineering, for steady guidance, timely suggestions and encouragement at every stage of this project.

We thank Head of Department name, Head of the Department of Computer Science and Engineering, for providing the facilities and the environment to carry out this work, and Principal / Dean name for constant support.

We are grateful to the project review committee for their feedback during the reviews, which shaped the Workshop Planner and the verification features, and to the Training and Placement Cell of our college for explaining how campus placement actually works, including the placement portal that inspired the placement profile.

We thank the faculty and staff of the department, our classmates who tried early versions of the system, and our families for their patience and encouragement.

V. Venkata Rupin
E. Manideep Reddy
J. Pruthvi Sai
S. Jaffer Ali

ABSTRACT

Campus placement portals record profiles, applications and results, but rarely tell anyone what to do next: which skill a student should learn, which students a recruiter should look at, or which training the placement cell should run. This project presents **CareerSphere AI**, a campus placement platform for students, recruiters and the placement cell built on one transparent, level-aware match rule: the weighted share of a job's required skills a student has, where must-have skills count twice as much as nice-to-have ones and a skill below the level asked earns half credit, with a 70% "within reach" threshold and the recruiter's CGPA cutoff. Levels can be verified by server-graded assessments or by certificates checked by the placement cell.

On this rule the system builds four decision tools. **Opportunity Math** ranks each possible learning step by how many open jobs it would bring within reach. A **Skill Roadmap** searches every bundle of up to three steps exactly and marks steps that only pay off together. **Hidden Talent** runs the rule from the job's side to show recruiters consented students who fit but have not applied. A **Workshop Planner** chooses k workshops for the batch by exact bit-mask search, because the objective is not submodular and greedy selection can fail. Two AI features use Google Gemini and are grounded in the same data: a Copilot that answers only through eight read-only tools, and a résumé skill finder that keeps a suggestion only if its quoted evidence occurs in the résumé; both work without an AI key.

The system is implemented with Spring Boot, React and MySQL, runs from Terminal, Xcode, VS Code and Android Studio and online with Docker, and has 39 unit tests and 17 end-to-end API tests. On a fictional batch of 35 students and 12 jobs, Opportunity Math brings 1.43 jobs per student within reach against 1.03 for the most-demanded missing skill; the Skill Roadmap raises this to 2.89 with two steps and 4.71 with three, helping all 7 students for whom no single skill helps; Hidden Talent surfaces 10 fitting non-applicants; and three exactly planned workshops give all 35 students a job within reach where greedy selection reaches 32.

Keywords: campus placement, skill-gap analysis, explainable recommendation, combinatorial optimisation, large language models, function calling, Spring Boot, React.

CONTENTS

Certificate ..... **i**
Declaration ..... **ii**
Acknowledgement ..... **iii**
Abstract ..... **iv**
List of Figures ..... **viii**
List of Tables ..... **ix**
List of Abbreviations ..... **x**
1 INTRODUCTION **1**
1.1 Background 1
1.2 Problem Statement 1
1.3 Motivation 2
1.4 Objectives 2
1.5 Scope of the Project 3
1.6 What Is New in This Project 3
1.7 Organisation of the Report 4
2 LITERATURE SURVEY **5**
2.1 Job Recommendation and Person–Job Fit 5
2.2 Placement Prediction 5
2.3 Explainable Recommendation 5
2.4 Coverage Problems and Submodularity 6
2.5 Tool Use and Grounding in Language Models 6
2.6 Fairness in Algorithmic Hiring 6
2.7 Existing Placement Portals 6
2.8 Research Gap 7
3 SYSTEM ANALYSIS AND REQUIREMENTS **8**
3.1 Existing System 8
3.2 Proposed System 8
3.3 Feasibility Study 8
3.3.1 Technical Feasibility 8
3.3.2 Economic Feasibility 9
3.3.3 Operational Feasibility 9
3.4 Functional Requirements 9
3.5 Non-Functional Requirements 10
3.6 Hardware and Software Requirements 10
3.7 Use Case Diagram 11
4 SYSTEM DESIGN **12**
4.1 System Architecture 12
4.2 Module Design 13
4.3 Database Design 14

4.4 Sequence Diagrams....16
4.4.1 Login....16
4.4.2 Applying to a Job....17
4.4.3 Asking the Copilot....18
4.4.4 Finding Skills in a Résumé....18
4.4.5 Building a Skill Roadmap....19
4.5 API Design....20
4.6 User Interface Design....20
4.7 Security Design....21
5 METHODOLOGY AND ALGORITHMS....22
5.1 Explainable Match Score....22
5.2 Within Reach....22
5.3 Opportunity Math....23
5.4 Skill Roadmap....23
5.5 Hidden Talent....24
5.6 Workshop Planner....24
5.6.1 Problem....24
5.6.2 Why Greedy Selection Fails....25
5.6.3 Exact Search with Bit Masks....25
5.7 Next Best Action....26
5.8 Résumé Score and Readiness Points....26
5.9 Skill Assessment Grading....27
5.10 Career Copilot....27
5.11 Evidence-Checked Résumé Skill Finder....28
6 IMPLEMENTATION....29
6.1 Technology Stack....29
6.2 Project Structure....30
6.3 Backend Implementation....31
6.4 Frontend Implementation....33
6.5 Screens....33
6.6 Running on Different Platforms....44
7 TESTING....45
7.1 Testing Strategy....45
7.2 Unit Tests....45
7.3 API Test Cases....46
7.4 User Interface Testing....47
7.5 Validation and Error Handling....47
8 RESULTS AND DISCUSSION....48
8.1 Demo Data....48
8.2 Which Skill to Learn Next....48
8.3 Skill Roadmap....48
8.4 Workshop Planning for the Batch....49
8.5 Hidden Talent....51

8.6 Résumé Skill Finder.....51
8.7 Human Evaluation Plan.....51
8.8 Limitations.....52
9 CONCLUSION AND FUTURE SCOPE.....53
9.1 Conclusion.....53
9.2 Future Scope.....53
References.....54
Appendix A: Installation and Running.....56
Appendix B: Demo Accounts and Demo Script.....57
Appendix C: Complete List of API Endpoints.....58

LIST OF FIGURES

3.1 Use case diagram of CareerSphere AI....11
4.1 System architecture of CareerSphere AI....13
4.2 Entity–relationship diagram (rotated to fit the page)....15
4.3 Sequence diagram: login and an authenticated request....17
4.4 Sequence diagram: applying to a job....18
4.5 Sequence diagram: Copilot with function calling....18
4.6 Sequence diagram: evidence-checked résumé skill finder....19
4.7 Sequence diagram: Skill Roadmap....19
5.1 Flow of the exact Workshop Planner....26
6.1 Mind map of the project: features, algorithms and technologies....30
6.2 Login page with demo accounts for the three roles....33
6.3 Career Map: each open job placed by match score; skill levers….....34
6.4 Career Map with the Copilot panel open....34
6.5 Opportunity Math: the skills that bring the most jobs within reach....35
6.6 Skill Roadmap: best one-, two- and three-step plans....35
6.7 Job listings with explained match scores....36
6.8 Job detail: why this score, and the job’s requirements....36
6.9 Placement profile: guided steps and résumé score (new student)....37
6.10 Placement profile of a student who has completed most steps....37
6.11 Skill assessments: verified and unverified skills....38
6.12 A timed assessment in progress....38
6.13 Assessment result: fewer than 4 correct verifies nothing, and a….....39
6.14 Certifications: a verified certificate and how verification works....39
6.15 Readiness leaderboard....40
6.16 Recruiter: posting a job with skill levels and must-have or….....40
6.17 Recruiter: applicants ranked by match, with verified skills....41
6.18 Recruiter: Hidden Talent for a job, with invitations....41
6.19 Placement cell: analytics and the curriculum–market skill gap....42
6.20 Placement cell: Workshop Planner for three workshops....42
6.21 Placement cell: certificates waiting for verification....43
6.22 The same app on a phone: Career Map and the slide-in menu....43
8.1 Outcome of each planning method for k = 1 to 4 workshops.….....50

LIST OF TABLES

2.1 Typical placement portal compared with CareerSphere AI.....7
3.1 Functional requirements by role.....9
3.2 Hardware and software requirements.....10
4.1 Modules of the system.....14
4.2 Main tables.....16
4.3 Main REST endpoints.....20
5.1 Résumé score rules.....27
6.1 Technology stack.....29
7.1 Unit tests.....45
7.2 End-to-end API test cases.....46
8.1 Recommended next skill, 35 students.....48
8.2 Skill Roadmap, 35 students.....49
8.3 Workshop plans: students reached / matches (Reach goal).....50
8.4 Planner runtime, median of 9 calls.....51
9.1 Demo accounts.....57
9.2 All REST endpoints.....58

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
BCrypt	Blowfish-based password hashing function
CGPA	Cumulative Grade Point Average
CSS	Cascading Style Sheets
DTO	Data Transfer Object
ER	Entity–Relationship
F1	Harmonic mean of precision and recall
H2	In-memory Java SQL database
HTTPS	Hypertext Transfer Protocol Secure
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MVC	Model–View–Controller
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
UI	User Interface
UUID	Universally Unique Identifier

CHAPTER 1
INTRODUCTION

1.1 Background

Campus placement is the main route from an engineering college to a first job in India. Within a few months each year, a training and placement cell invites companies, students apply to openings, recruiters shortlist and interview, and offers are made. The process is high in volume and short in time. A mid-sized department may have several hundred final-year students and dozens of openings, each asking for its own mix of technical skills, a minimum CGPA and sometimes a particular branch.

Most colleges run this process on a placement portal: a website where students keep a profile and résumé, companies post drives, and the placement cell tracks applications. These portals are good at recording what happened. They are much weaker at telling anybody what to do next. A student can see that a job asks for Java, Spring Boot and Git, but not whether learning Spring Boot or Docker would open more doors across all current openings. A recruiter sees a list of applicants, but not the students who would fit well and simply did not apply. A placement cell can see that many students lack a skill, but not which three workshops would help the most students reach at least one job.

This project, **CareerSphere AI**, is a campus placement platform built around one transparent rule: how much of a job's required skill set a student has. Because the rule is simple and exact, it can be run thousands of times a second to answer "what if" questions for students, recruiters and the placement cell, and every answer can be explained line by line.

1.2 Problem Statement

Existing placement systems either show a score and a list of missing skills, or predict whether a student will be placed. Neither tells a student which action to take, and neither helps the placement cell plan training for the whole batch. The problem addressed in this project is therefore:

> *To design and build a campus placement platform that (i) scores every student against every open job with a transparent, level-aware rule; (ii) turns that score into concrete, explainable decisions: which skill or set of skills a student should learn next, which students a recruiter should look at, and which workshops a placement cell should run, for whom; and (iii) uses large language models only where their answers can be grounded in the same data, so that the system never invents a skill or a number.*

1.3 Motivation

Three observations motivated the project. First, skills pay off in combination. A student who has only Git gains nothing from learning Java alone or Spring Boot alone for a job that needs all three, but gains the job by learning both. Advice that looks at one skill at a time can therefore be wrong in a way that matters. Second, placement data is small enough to search exactly. A batch of 35 students, 12 jobs and 22 skills has only 1,540 ways to choose three workshops, so the best plan can be found in milliseconds rather than approximated. Third, large language models are good at conversation and at reading résumés, but they can state things that are not true. A placement decision should not depend on such a statement unless it can be checked.

1.4 Objectives

1. Build a working web platform for three roles (student, recruiter, placement cell) with secure login, profiles, résumé upload, job posting, applications and status tracking.
2. Define an explainable, level-aware match score with must-have and nice-to-have skills, and a clear “within reach” rule that includes the recruiter’s CGPA cutoff.
3. Give every student actionable guidance: Opportunity Math (the best next skill), a what-if Career Map, and a Skill Roadmap that finds the best two- and three-step plans exactly.
4. Give recruiters ranked, explained applicants with verified skills, and a Hidden Talent list of consented students who fit but have not applied.
5. Give the placement cell batch analytics, certificate verification, a readiness leaderboard and a Workshop Planner that chooses the best k workshops by exact search.
6. Verify skill levels through server-graded assessments and placement-cell-checked certificates.
7. Add an AI Copilot and an AI résumé skill finder that are grounded in the student’s real data and keep working without an AI key.
8. Test the system with automated unit and API tests, and evaluate the methods on a realistic demo batch against simple baselines.
9. Make the project easy to run and demonstrate: on a Mac with one command, from Xcode, VS Code and Android Studio, and online with Docker.

1.5 Scope of the Project

The system covers the core placement cycle inside one college: profiles, résumés, skills and levels, jobs, applications, shortlisting and offers, plus the guidance and planning features described above. It does not handle interview scheduling with calendars, payroll or offer letters, or communication with companies outside the platform. All numbers in this report come from a fictional demo batch shipped with the system (35 students, 2 companies, 12 jobs, 22 skills), so they show how the methods behave rather than their effect on real placements.

1.6 What Is New in This Project

Several features are, to our knowledge, not found together in college placement portals or in the published systems reviewed in Chapter 2:

- **Opportunity Math.** For each missing skill or level-up, the system re-scores every open job and counts how many would newly come within reach. The student sees “Learn Spring Boot: +4 jobs”, not just “you are missing Spring Boot”.
- **Skill Roadmap with synergy detection.** An exact search over every bundle of up to three learning steps finds the best plan of each size and marks bundles whose steps only pay off together. On the demo batch this raises the average gain from 1.43 jobs (best single skill) to 4.71 jobs (best three steps), and helps all 7 students for whom no single skill helps.
- **Hidden Talent.** The same rule run from the job’s side shows a recruiter consented students who fit a job, or are one step away, without having applied, and lets the recruiter invite them.
- **Exact Workshop Planner.** Choosing workshops for a batch is shown not to be submodular, so greedy selection can fail. Exact search with bit masks finds the best plan; with three workshops it gives all 35 demo students a job within reach, where greedy selection reaches 32.
- **Grounded AI.** The Copilot answers only through eight read-only tools over the student’s own data, and the résumé skill finder keeps an AI suggestion only if its quoted evidence is really in the résumé.
- **Verified levels.** Skill levels can be verified by a server-graded assessment or a certificate checked by the placement cell, and verified levels are what recruiters and the score use.

1.7 Organisation of the Report

Chapter 2 reviews related work and existing placement portals. Chapter 3 analyses the requirements. Chapter 4 presents the system design: architecture, database, sequence diagrams and APIs. Chapter 5 describes the methods and algorithms in detail. Chapter 6 covers the implementation, with screenshots of every main screen. Chapter 7 describes testing, Chapter 8 the results, and Chapter 9 concludes with future scope. The appendices give the installation guide, the demo script and the full list of APIs.

CHAPTER 2
LITERATURE SURVEY

This chapter reviews work in five areas that the project builds on: job recommendation and person–job fit, placement prediction, explainable recommendation, coverage problems and submodularity, and tool use and grounding in large language models, followed by fairness in algorithmic hiring and a comparison with existing placement portals.

2.1 Job Recommendation and Person–Job Fit

De Ruijt and Bhulai [1] review a decade of job recommender systems. They group methods into content-based, collaborative, hybrid and knowledge-based approaches and point out that job recommendation differs from product recommendation: it is reciprocal (both the candidate and the employer must agree), temporal (openings close), and has received limited attention to fairness. Zhu et al. [2] learn joint neural representations of résumés and job postings to predict person–job fit. Such models can be accurate when large labelled histories exist, but a college placement cell rarely has that data, and a learned score offers little explanation of why a particular student fits a particular job.

2.2 Placement Prediction

Many studies predict whether a student will be placed. Alfana et al. [3], for example, build probability models from cognitive and academic features to predict campus placement outcomes. Prediction is useful for a college's planning, but it describes a situation rather than changing it: knowing that a student has a 40% chance of placement does not say which skill to learn to improve that chance. CareerSphere instead computes the effect of each possible action exactly.

2.3 Explainable Recommendation

Zhang and Chen [4] survey explainable recommendation and distinguish *post-hoc* explanations, generated after a black-box model makes a prediction, from models that are *interpretable by construction*. CareerSphere is of the second kind. Every score is computed from the listed skills and levels, so the explanation (matched skills, missing skills, level gaps, CGPA rule) is the computation itself rather than an approximation of it. A student or recruiter can check any score by hand.

2.4 Coverage Problems and Submodularity

Choosing k items to cover as many users as possible is a maximum coverage problem. For monotone submodular objectives, where adding an item helps less the more items have already been chosen, the greedy algorithm achieves a $1 - 1/e$ (about 63%) approximation [5], which is the best possible in polynomial time unless P = NP [6]. Krause and Golovin [7] survey the area. The workshop planning problem in this project has a threshold (a job is within reach at 70% of its skills), which creates complementarities between skills and breaks submodularity, so the greedy guarantee does not apply. Section 5.7 gives a minimal example and Chapter 8 shows greedy selection failing on the demo batch.

2.5 Tool Use and Grounding in Language Models

Toolformer [8] and ReAct [9] show that language models can decide when to call external tools and use the results in their answers; commercial APIs expose the same pattern as *function calling* [16]. Ji et al. [10] survey hallucination in natural language generation, where a model states things not supported by its input, and Gao et al. [11] show that even strong models often fail to support their statements with correct citations. CareerSphere restricts the Copilot to read-only tools over the student's own data and verifies résumé evidence by exact string matching, so unsupported claims are dropped rather than shown.

2.6 Fairness in Algorithmic Hiring

Raghavan et al. [12] examine the claims made by vendors of algorithmic pre-employment assessments and the practical limits of removing bias. CareerSphere uses no demographic attributes. Its only inputs are skills, levels and the recruiter's published CGPA cutoff, every score can be audited by hand, and a student must give explicit placement consent before recruiters can see the profile.

2.7 Existing Placement Portals

College placement portals, including the one used at our university, typically offer student profiles, résumé upload, placement consent, skill assessments, certificate upload with verification, an introduction video, drive listings and a readiness or résumé score. CareerSphere includes these features and adds the decision-support layer. Table 2.1 summarises the comparison.

Table 2.1: Typical placement portal compared with CareerSphere AI

Feature	Typical portal	CareerSphere AI
Profile, résumé, consent, video	Yes	Yes, as guided steps
Skill assessment	Often	Yes, server-graded; sets a verified level
Certificate verification	Often	Yes, by the placement cell
Match score for a job	Rarely, or opaque	Yes, explained line by line, level-aware
Best next skill across all jobs	No	Opportunity Math
Best 2- and 3-step learning plan	No	Skill Roadmap (exact search)
Recruiter sees fitting non-applicants	No	Hidden Talent with invitations
Workshop planning for the batch	No	Exact Workshop Planner
AI assistant	Rarely	Copilot grounded in 8 read-only tools
AI résumé reading	Rarely	Evidence-checked skill finder

2.8 Research Gap

The literature offers accurate but opaque fit models, outcome predictors, and general results on coverage and on tool-using language models. What is missing for a college is a system that connects a transparent score to decisions at three levels (student, recruiter, placement cell), recognises that skills pay off in combination, and uses language models only where their output can be checked. CareerSphere is designed to fill this gap.

CHAPTER 3
SYSTEM ANALYSIS AND REQUIREMENTS

3.1 Existing System

In the existing process a student fills a profile and uploads a résumé on the placement portal, browses drives, and applies. The placement cell exports applicant lists to spreadsheets for recruiters and plans training sessions from experience. The drawbacks are:

- Students do not know which jobs they are close to, or which skill would help the most.
- Recruiters screen applicants by reading résumés, and never see good students who did not apply.
- Skill claims are self-reported and rarely verified.
- Training plans are chosen by intuition, without measuring how many students each plan would help.
- Any AI feature, if present, can produce statements nobody can check.

3.2 Proposed System

CareerSphere keeps the familiar portal workflow and adds a decision layer on top of a single transparent match rule. Students see every open job on a Career Map placed by match score and can try skills on the map before learning them. Opportunity Math and the Skill Roadmap say exactly which skills to learn. Recruiters see ranked applicants with verified skills and a Hidden Talent list. The placement cell verifies certificates, sees a readiness leaderboard and plans workshops with an exact planner. A Copilot answers questions from the student’s own data.

3.3 Feasibility Study

3.3.1 Technical Feasibility

All components use mature, free and widely taught technologies: Java 17 with Spring Boot, React, and MySQL. The exact searches are small (about 1,000 bundles per student, 1,540 workshop plans for k = 3) and run in milliseconds on a laptop. The AI features use the Gemini API’s free tier and fall back to rule-based answers without a key.

3.3.2 Economic Feasibility

The software stack has no licence cost. Development and demonstration run on a student laptop. Online hosting on a platform such as Railway costs a few dollars a month for a small college, and the system can equally run on a college server with Docker.

3.3.3 Operational Feasibility

The workflow follows the steps students and placement officers already know from existing portals, and every screen explains its numbers in plain language. Demo accounts and seeded data let a new user explore all three roles immediately.

3.4 Functional Requirements

Table 3.1: Functional requirements by role

ID	Role	Requirement
FR-1	All	Register (student or recruiter), log in and log out; admin accounts cannot be self-registered.
FR-2	Student	Maintain a profile: branch, CGPA, marks, links, headline, skills with levels.
FR-3	Student	Upload a résumé (PDF, DOC, DOCX up to 5 MB); have skills suggested from it with evidence.
FR-4	Student	Give or withdraw placement consent; record or upload a one-minute introduction video.
FR-5	Student	Take a timed skill assessment; upload certificates for verification.
FR-6	Student	See every open job with its match score and explanation; apply once per job.
FR-7	Student	See the Career Map, Opportunity Math, what-if results and the Skill Roadmap.
FR-8	Student	Track application status; see recruiter invitations; see the readiness leaderboard.
FR-9	Student	Ask the Copilot questions about jobs, skills and applications.
FR-10	Recruiter	Maintain a company profile; post, edit, close and delete jobs with skill levels and must-have or nice-to-have flags.
FR-11	Recruiter	See applicants ranked by match with verified skills, certificates, résumé and video; update status.
FR-12	Recruiter	See Hidden Talent for a job and invite students to apply.
FR-13	Admin	See placement analytics, the skill demand–supply gap and the batch leaderboard.
FR-14	Admin	Verify or reject certificates with a note.
FR-15	Admin	Plan k workshops for a goal and see who to invite to each.

3.5 Non-Functional Requirements

- **Security:** passwords stored as BCrypt hashes; stateless JWT authentication; every API restricted to one role on the server; ownership checks so a recruiter sees only their own applicants; uploads checked by size and file signature.
- **Privacy:** recruiters see a student only after placement consent; the leaderboard shows other students by first name and initial; the AI key stays on the server.
- **Performance:** every page answers in well under a second on the demo data; the exact searches take milliseconds.
- **Explainability:** every number shown has a sentence explaining how it was computed.
- **Reliability:** the system works fully without an AI key; duplicate applications and invitations are prevented by database constraints.
- **Usability and portability:** responsive layout down to phone width; runs in any modern browser, as a Mac and iPhone app, and as an Android app.
- **Maintainability:** layered code (controller, service, repository), validation on the server, automated tests.

3.6 Hardware and Software Requirements

Table 3.2: Hardware and software requirements

Item	Minimum	Used in development
Processor	Dual-core, 2 GHz	Apple M-series / Intel Core i5
Memory	4 GB RAM	8 GB RAM or more
Disk	2 GB free	5 GB free
Operating system	macOS, Windows or Linux	macOS 14 or later
Java	JDK 17	Temurin JDK 17 or 21
Node.js	Node 18	Node 20 LTS
Database	MySQL 8	MySQL 8.4 (Homebrew)
Build tools	Maven 3.9, npm	Maven (Homebrew), npm
Editors (optional)	Any	VS Code, Xcode 15+, Android Studio
Browser	Chrome, Safari, Edge, Firefox	Chrome, Safari
AI (optional)	Gemini API key	Gemini free tier, demo data only

3.7 Use Case Diagram

Figure 3.1 shows the main use cases of the three actors. Students manage their placement profile and use the guidance features; recruiters post jobs and review applicants and hidden talent; the placement cell verifies certificates, views analytics and plans workshops. The Gemini service is an external actor used by the Copilot and the résumé skill finder.

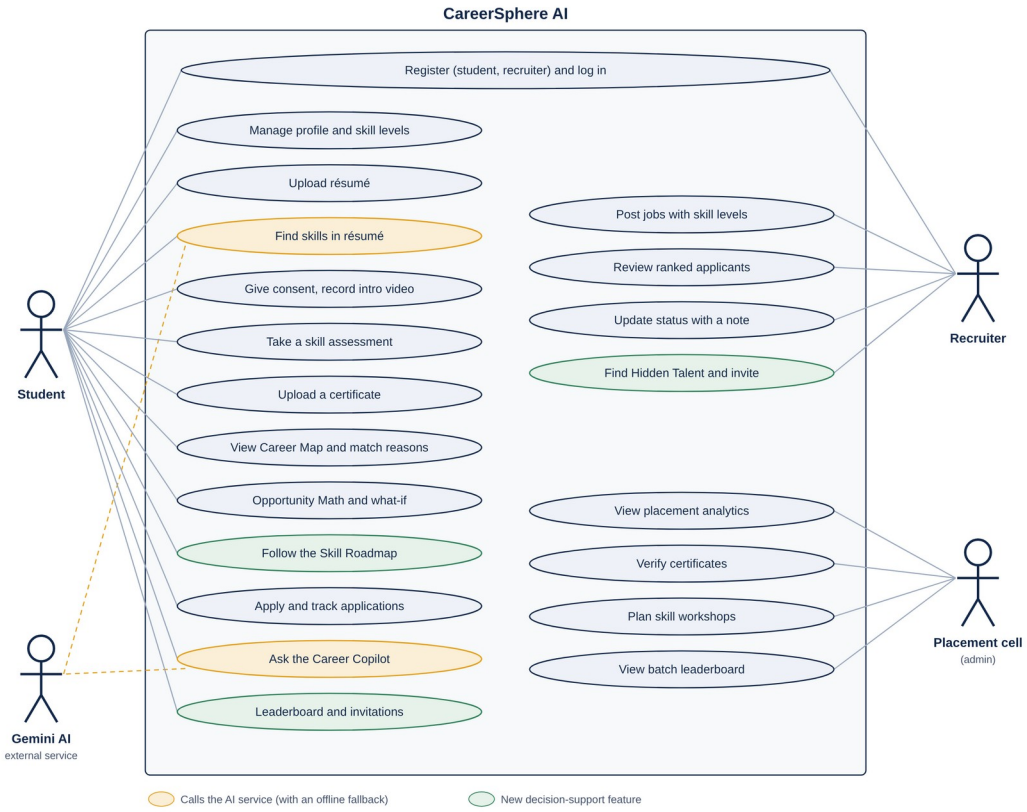


Figure 3.1: Use case diagram of CareerSphere AI

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

CareerSphere follows a three-tier client–server architecture (Figure 4.1). The client is a single-page React application, used directly in a browser or wrapped by the Mac, iPhone and Android apps. The server is a Spring Boot application that exposes a REST API under `/api`. Every request passes through Spring Security, which validates the JSON Web Token and the user's role before a controller is reached. Controllers are thin: they call services, which hold the business rules, and services read and write MySQL through Spring Data JPA repositories. Uploaded files are stored on disk under random names.

Two layers deserve special mention. The **rule engine** (MatchService, RoadmapService, TalentService and WorkshopService) computes every score and plan from the same match rule, so a student, a recruiter and the placement cell always see consistent numbers. The **AI layer** never writes data on its own: the Copilot reads data only through tools that call the same services, and the résumé skill finder only proposes skills that the student must confirm. The Gemini API key is held by the server and never sent to the browser.

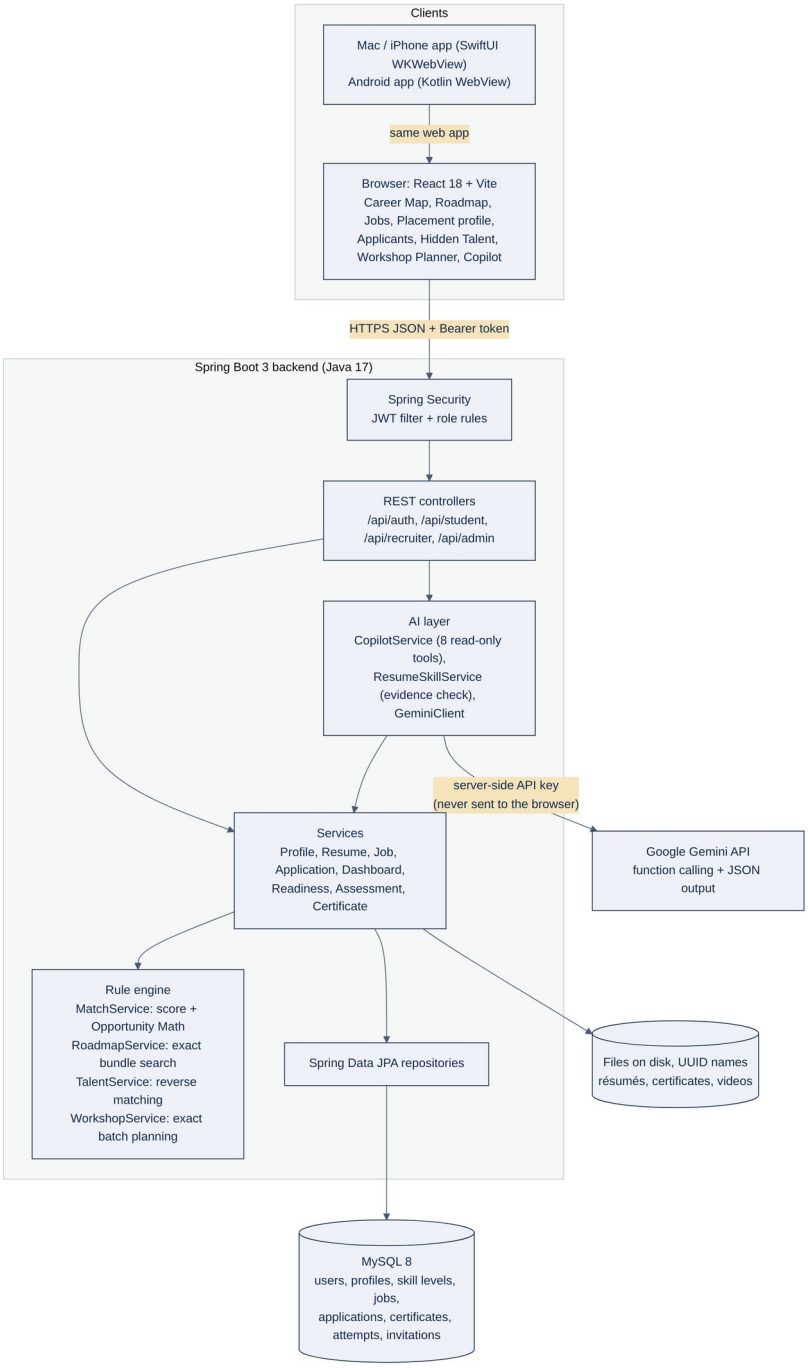


Figure 4.1: System architecture of CareerSphere AI

4.2 Module Design

The system is divided into the modules listed in Table 4.1. Each module has its own controller endpoints, services and pages, and all modules share the match rule.

Table 4.1: Modules of the system

Module	Main responsibilities	Main classes and pages
Authentication	Register, log in, JWT issue and check, role rules	AuthService, JwtService, SecurityConfig; Login, Register
Student profile	Profile, skills and levels, résumé, consent, video, résumé score	StudentProfileService, ResumeService, ReadinessService; PlacementProfile
Skill verification	Timed assessments, certificate upload and review	AssessmentService, QuestionBank, CertificateService; Assessments, Certifications
Jobs and applications	Post jobs with levels, apply, track status, ranked applicants	JobService, ApplicationService; StudentJobs, JobForm, Applicants
Guidance	Match score, Career Map, Opportunity Math, what-if, Skill Roadmap, next best action	MatchService, DashboardService, RoadmapService; CareerMap, Opportunities, Roadmap
Hidden Talent	Consented non-applicants who fit; invitations	TalentService, Invitation; Applicants (Hidden talent card)
Placement cell	Analytics, workshop planning, certificate review, leaderboard	WorkshopService, AdminController; AdminDashboard, WorkshopPlanner
AI	Copilot with tools; résumé skill finder	CopilotService, CopilotTools, ResumeSkillService, GeminiClient

4.3 Database Design

Figure 4.2 shows the entity–relationship diagram. The `users` table holds every account with a role. Students and recruiters each have a one-to-one profile. Skills are a shared catalogue. A student's skills are stored with a level (1 Beginner, 2 Intermediate, 3 Advanced) and, separately, the level that has been verified. A job's skills are stored with a minimum level and a must-have or nice-to-have flag. Applications and invitations each link one job and one student, with a unique constraint on the pair, so a student cannot apply twice and a recruiter cannot invite the same student twice to the same job even under simultaneous requests. Certificates and assessment attempts belong to a student and a skill.

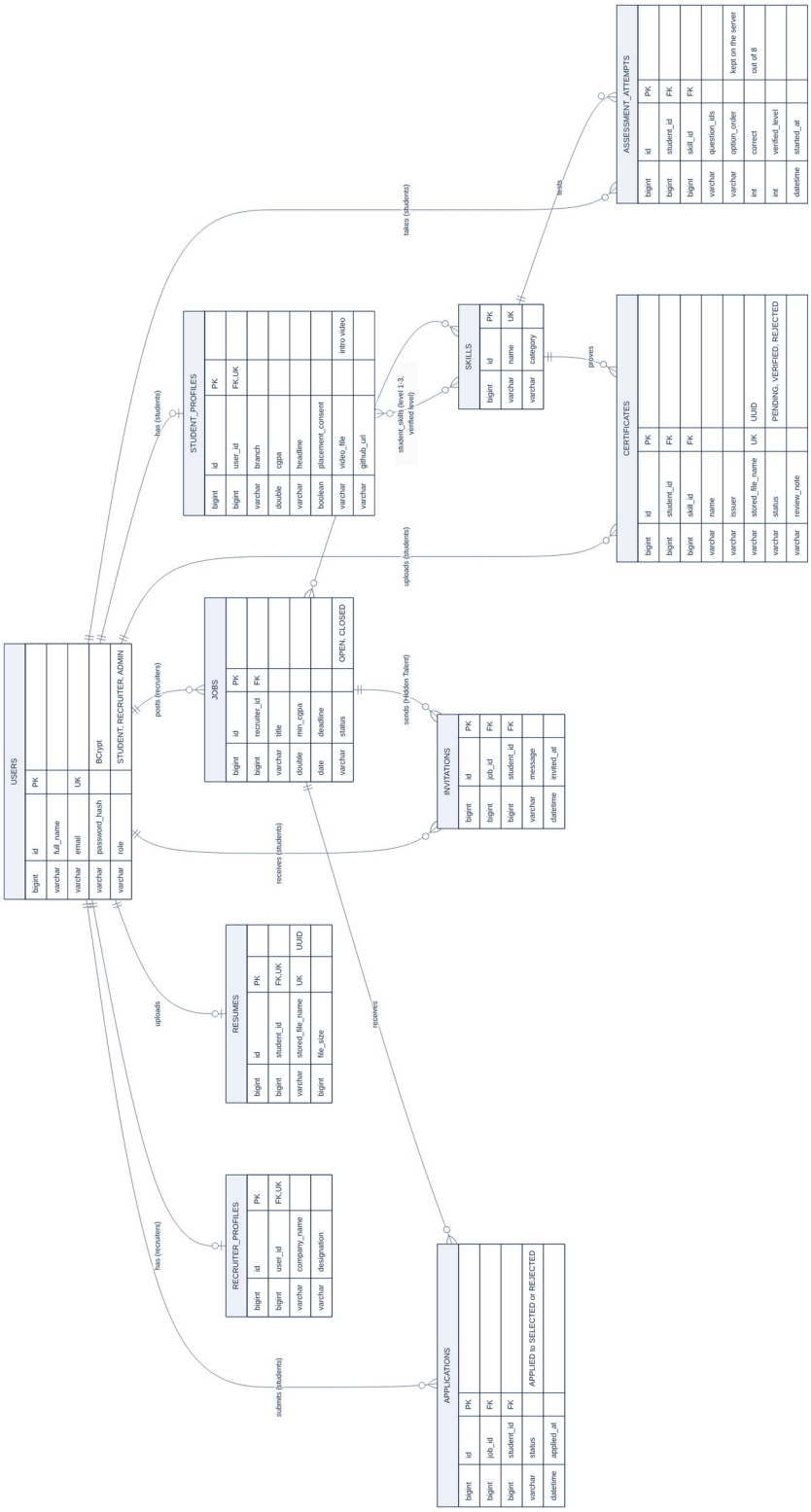


Figure 4.2: Entity–relationship diagram (rotated to fit the page)

Table 4.2: Main tables

Table	Purpose	Key constraints
users	All accounts with role STUDENT, RECRUITER or ADMIN	email unique; password stored as BCrypt hash
student_profiles	Academic details, links, headline, consent, video	user_id unique (one profile per student)
student_skills	Skill and self-reported level per student	level 1 to 3
student_verified_skills	Verified level per student and skill	set only by assessment or certificate
jobs / job_skills	Openings and their skills with minimum level and weight	status OPEN or CLOSED
applications	A student’s application to a job and its status	(job_id, student_id) unique
resumes	One résumé file per student	student_id unique; stored under a UUID name
certificates	Uploaded certificates and their review	status PENDING, VERIFIED or REJECTED
assessment_attempts	Questions, option order, result of each test	one running attempt per student
invitations	Recruiter invitations from Hidden Talent	(job_id, student_id) unique

4.4 Sequence Diagrams

The following sequence diagrams show how the main operations travel through the layers.

4.4.1 Login

The user submits an email and password; AuthService loads the user, checks the BCrypt hash and issues a signed token valid for eight hours, carrying the user id and role (Figure 4.3). Later requests send the token in the `Authorization` header and the JWT filter rebuilds the user from it.

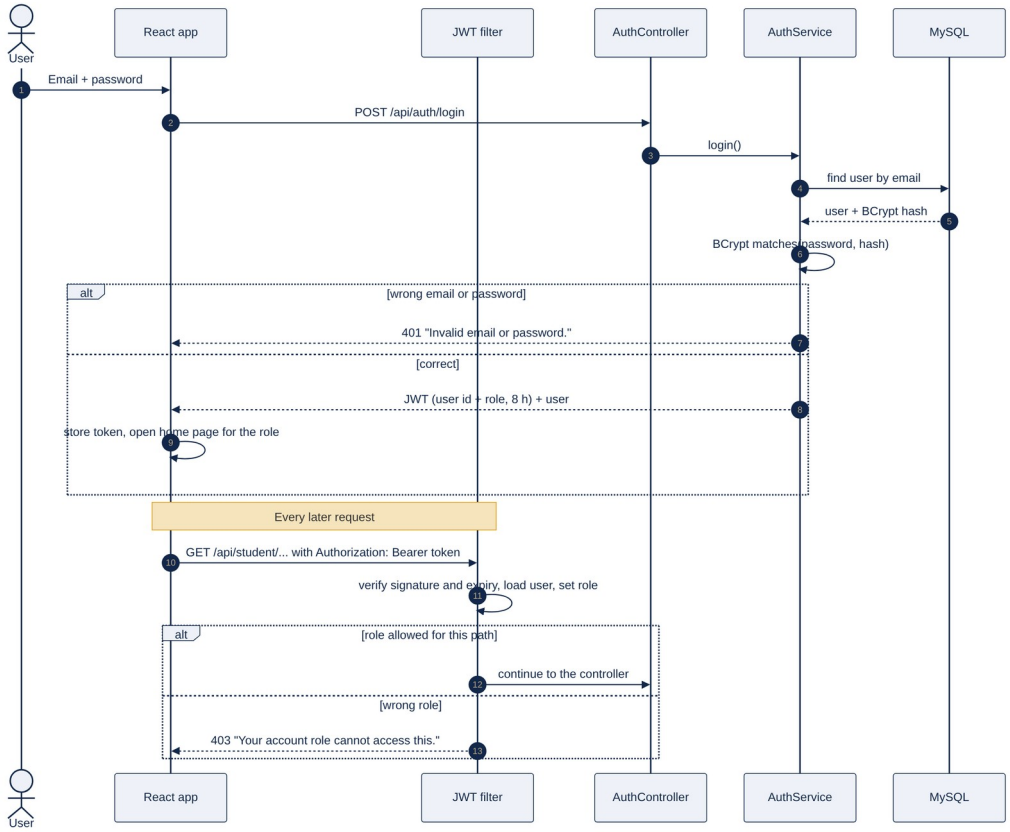


Figure 4.3: Sequence diagram: login and an authenticated request

4.4.2 Applying to a Job

Applying checks, in order, that the job is open and before its deadline, that the student has a résumé and has given placement consent, that the CGPA meets any cutoff, and that the student has not already applied (Figure 4.4). The unique database constraint is the final guard.

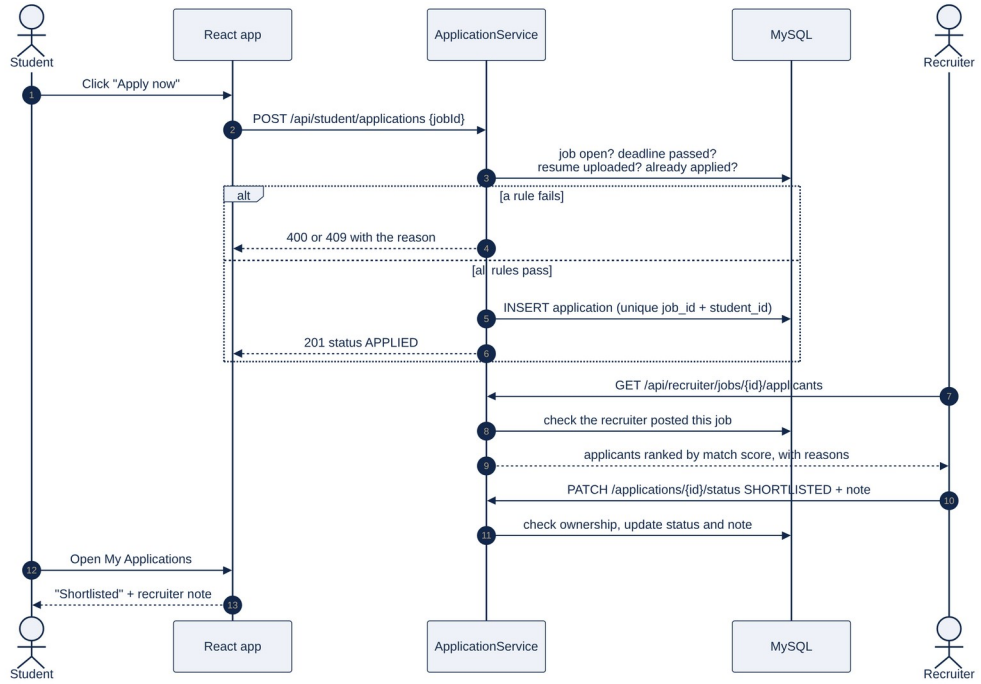


Figure 4.4: Sequence diagram: applying to a job

4.4.3 Asking the Copilot

The Copilot sends the conversation and the tool definitions to Gemini. When the model asks for a tool, the server runs it for the logged-in student and returns the result; this repeats at most five times before the model writes the final answer (Figure 4.5). Without a key, the offline mode answers from the same tools.

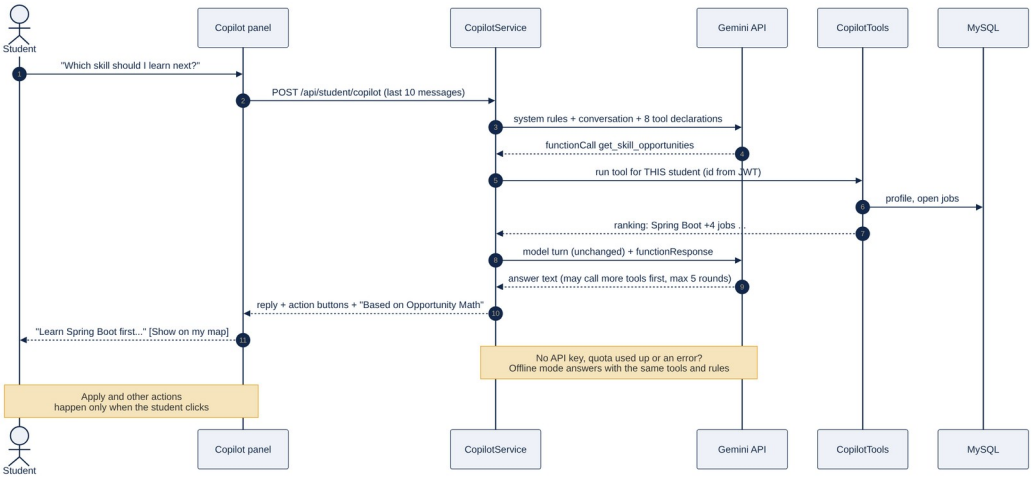


Figure 4.5: Sequence diagram: Copilot with function calling

4.4.4 Finding Skills in a Résumé

The résumé text is extracted, Gemini returns candidate skills with a quoted line each, and the server keeps a suggestion only if the quote really occurs in the text and the skill is in the catalogue (Figure 4.6). The student then confirms which suggestions to add.

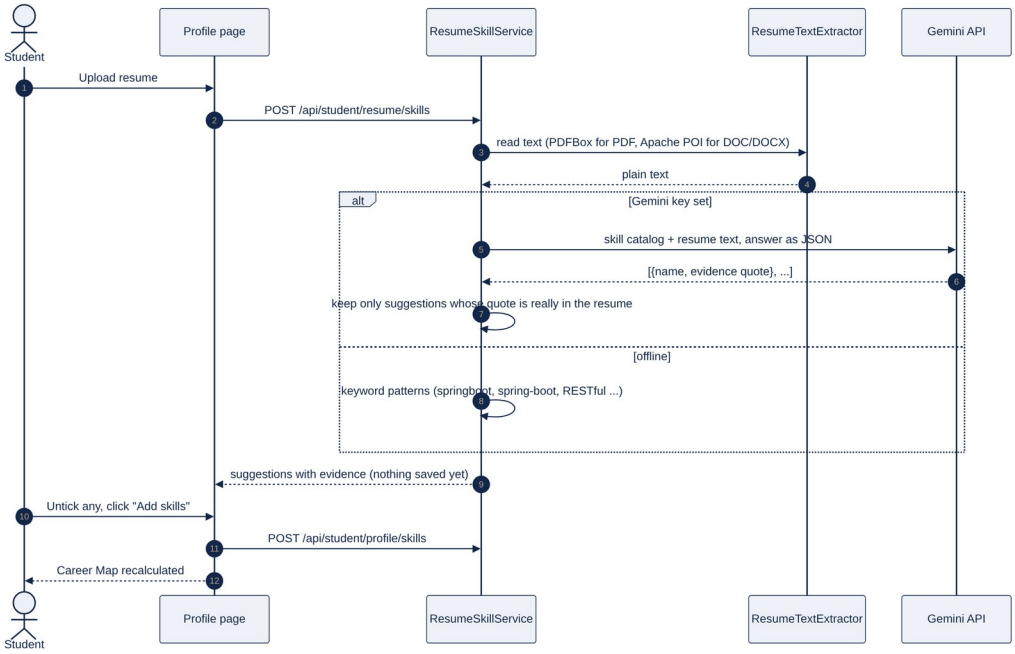


Figure 4.6: Sequence diagram: evidence-checked résumé skill finder

4.4.5 Building a Skill Roadmap

The roadmap is computed on request: RoadmapService loads the student's levels, CGPA and the open jobs, generates every learning action, checks every bundle of up to three actions against every job, and returns the best bundle of each size with the cheapest path to each nearby job (Figure 4.7).

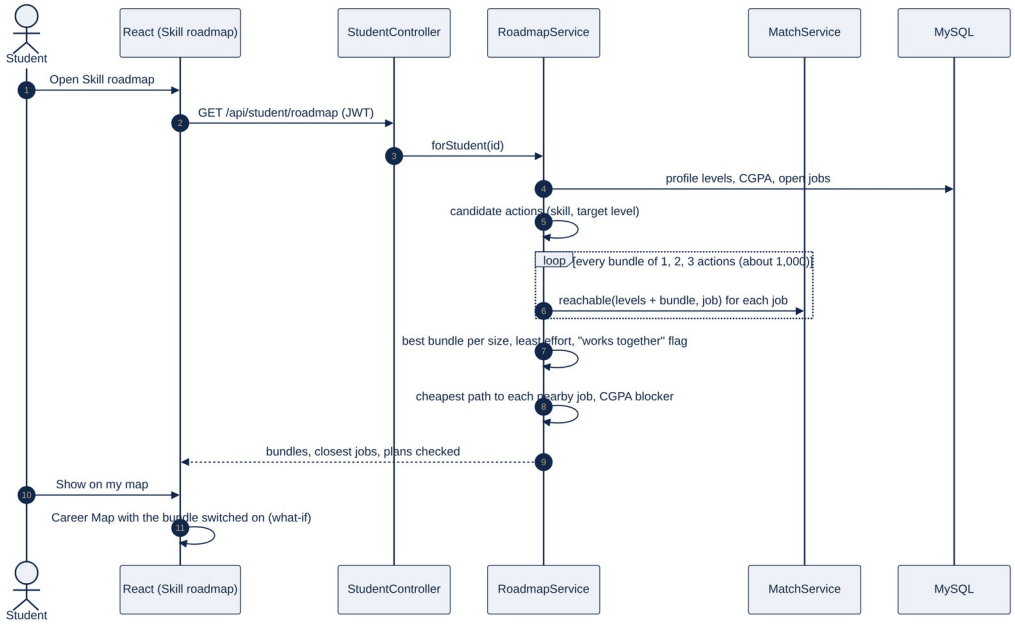


Figure 4.7: Sequence diagram: Skill Roadmap

4.5 API Design

The REST API is grouped by role. All endpoints except register, login, health and the skill catalogue need a valid token, and each prefix is open to one role only. Requests and responses are JSON, except file uploads (multipart) and file downloads. Errors return a status code with a plain-language message and, for forms, a message per field. Table 4.3 lists the main endpoints; Appendix C lists all of them.

Table 4.3: Main REST endpoints

Method and path	Role	Purpose
POST /api/auth/register, /login	Public	Create an account; log in and receive a token
GET /api/student/dashboard	Student	Career Map data, top matches, next best action
GET /api/student/opportunities	Student	Opportunity Math ranking
GET /api/student/what-if?skills=	Student	Re-score all jobs with extra skills or levels
GET /api/student/roadmap	Student	Best 1-, 2-, 3-step bundles and paths to jobs
POST /api/student/applications	Student	Apply to a job
POST /api/student/assessments/start	Student	Start a timed assessment for a skill
POST /api/student/certificates	Student	Upload a certificate for verification
POST /api/student/copilot	Student	Ask the Copilot
POST /api/recruiter/jobs	Recruiter	Post a job with skill levels
GET /api/recruiter/jobs/{id}/applicants	Recruiter	Ranked, explained applicants
GET /api/recruiter/jobs/{id}/talent	Recruiter	Hidden Talent for a job
POST /api/recruiter/jobs/{id}/invite/{sid}	Recruiter	Invite a student to apply
GET /api/admin/workshops? rounds=GET /api/admin/workshops? k=&goal=goal=	Admin	Workshop plan
PATCH /api/admin/certificates/{id}	Admin	Verify or reject a certificate

4.6 User Interface Design

The interface uses a fixed sidebar grouped like a placement portal (Overview, Placements, Growth, Account for students; Hiring for recruiters; Placement cell for the admin), a top bar with search and the user’s name, and a content area. A deep navy and saffron palette is used throughout, with large, readable numbers for the values that matter. On a phone the sidebar becomes a slide-in menu. Every number is accompanied by a sentence saying how it was computed, and every destructive action asks for confirmation.

4.7 Security Design

- **Authentication:** stateless JWT signed with an HMAC-SHA key, eight-hour expiry; passwords hashed with BCrypt [14].
- **Authorisation:** URL prefixes restricted by role in SecurityConfig, plus ownership checks in services (a recruiter can update only applications to their own jobs).
- **Admin accounts** cannot be created through the public register endpoint.
- **Uploads:** size limits (résumé 5 MB, certificate 5 MB, video 25 MB), allowed types checked against the file's first bytes (PDF and Word for résumés; PDF, PNG, JPEG for certificates; WebM, MP4 for videos), random UUID file names, no user-controlled paths.
- **Assessments:** answers never leave the server; the option order is stored with the attempt; time is checked on the server.
- **Privacy:** recruiters see students only after consent; other students on the leaderboard appear by first name and initial; the AI key is a server environment variable.
- **Validation:** every request body is validated with Bean Validation and returns field-level errors.

CHAPTER 5
METHODOLOGY AND ALGORITHMS

This chapter defines the match rule and every method built on it. All methods use integer arithmetic so that the planner, the roadmap and the explanation shown to the user never disagree by rounding.

5.1 Explainable Match Score

Let a student s have level $\ell_s(x) \in \{0, 1, 2, 3\}$ for each skill x (0 means not known; 1 to 3 are Beginner, Intermediate and Advanced), and CGPA g_s. Let an open job j require the skill set R_j, with a minimum level $m_j(x)$ for each skill, a weight $w_j(x) = 2$ for a must-have skill and 1 for a nice-to-have skill, and an optional CGPA cutoff c_j. The credit for one skill is

$$\kappa(\ell, m) = 1 \text{ if } \ell \geq m; \quad \tfrac{1}{2} \text{ if } 0 < \ell < m; \quad 0 \text{ if } \ell = 0 \tag{5.1}$$

and the match score is the weighted share of credit:

$$\text{score}(s, j) = \text{round}\left(100 \cdot \Sigma_x \, w_j(x) \cdot \kappa(\ell_s(x), m_j(x)) \,/\, \Sigma_x \, w_j(x) \right) \tag{5.2}$$

with both sums over the required skills $x \in R_j$. A skill below the level asked therefore earns half credit and is reported as a *level gap*. In code the sums are kept in half units, `half` and `total`, and the rounded percentage is computed as (200 · half + total) div (2 · total), which rounds halves up without floating point. When every skill is a must-have at Beginner level, (5.2) reduces to the plain share of required skills, $|H_s \cap R_j| \,/\, |R_j|$; the demo jobs use these defaults.

Worked example. A job needs Java at Intermediate (must-have), Spring Boot (must-have) and Git (nice-to-have). A student has Java at Beginner, Spring Boot and Git. Credit in half units: Java 2 (half of 4), Spring Boot 4, Git 2; total 10. Score = (200 · 8 + 10) div 20 = 80%. The explanation reads: “You have 3 of 3 required skills. Below the level asked (half credit): Java (you: Beginner, needs Intermediate).”

5.2 Within Reach

$$\text{reach}(s, j) = [\ \text{score}(s, j) \geq 70\ ] \wedge [\ g_s \geq c_j\ ] \tag{5.3}$$

A job is *within reach* when (5.3) holds. With default jobs the 70% threshold admits 3 of 4 and 4 of 5 required skills but not 2 of 3 (67%). The CGPA rule is reported separately and never changes the score, so a student can see that a job is a good skill match but closed to them, and why.

5.3 Opportunity Math

An *action* is learning a skill x to a level t, or raising a known skill to a level t that some open job asks for. Writing L_s for the student's levels and $L_s \oplus (x, t)$ for the levels after the action, the gain of an action is the number of open jobs that newly come within reach, where reach(L, j) is (5.3) evaluated with levels L:

$$\text{gain}_s(x, t) = |\{\, j : \neg\text{reach}(L_s, j) \wedge \text{reach}(L_s \oplus (x, t), j) \,\}| \tag{5.4}$$

For each skill the target level with the largest gain (then the lowest level) is kept, so the student sees either "Learn Docker" or "Level up Java to Advanced". Actions are ranked by gain, with ties broken by the number of open jobs requiring the skill. The **what-if simulator** applies the same rule to any set of actions; on the Career Map each job is drawn at distance 100 − score from the student, so toggling a skill lever moves jobs across the 70% ring.

5.4 Skill Roadmap

Opportunity Math looks one step ahead, but skills pay off together. A student with only Git gains nothing from Java alone or Spring Boot alone for a job that needs all three (2 of 3 is 67%), yet gains the job by learning both. The Skill Roadmap therefore searches exactly over bundles B of 1, 2 and 3 actions, with at most one action per skill, and for each bundle size keeps the bundle that maximises the number of jobs within reach and then minimises the effort

$$\text{effort}(B) = \Sigma_{(x,\, t) \in B} \left(t - \ell_s(x) \right) \tag{5.5}$$

that is, the number of levels gained. A bundle is shown only if it beats every smaller one. It is marked **works together** when its gain is larger than the sum of the single gains (5.4) of its steps, which is exactly the case where one-at-a-time advice fails. Steps are ordered by their immediate effect, and the page shows how many jobs are within reach after each step. For every nearby job the roadmap also finds the cheapest subset of that job's own gaps that brings it within reach, or names the CGPA cutoff when no skill can help (Algorithm 5.1).

Algorithm 5.1: Skill Roadmap for one student

```
input: levels L, CGPA g, open jobs J
A ← every (skill, target level) some job asks for above L
now ← jobs within reach with L
for k ← 1 to 3 do
    best ← bundle of k actions (one per skill) with most jobs within reach, then least effort
    if reach(best) > reach of every smaller bundle shown then
        order steps by immediate effect; mark works together if gain > sum of single gains
        add best to the roadmap
for each job j not within reach do
    if g < cutoff(j) then report the CGPA blocker
    else cheapest subset of j's gaps that brings j within reach
return bundles, paths to the closest jobs, number of bundles checked
```

With about 30 candidate actions a student needs roughly 10^3 bundle checks (1,083 on average on the demo batch), each a pass over 12 jobs, so the roadmap is computed fresh on every page load.

5.5 Hidden Talent

Hidden Talent runs the match rule from the job's side. For a recruiter's job it considers every student who has given placement consent and has not applied to that job, and lists (i) those for whom the job is already within reach and (ii) those for whom a single action from Section 5.3 would bring it within reach, naming the cheapest such action. Both lists are sorted by score. The recruiter can send an invitation with an optional message, which appears on the student's Jobs page. Students without consent never appear, so the feature widens the recruiter's pool without weakening the consent rule.

5.6 Workshop Planner

5.6.1 Problem

Let S be the students, J the open jobs and C the candidate skills (every skill some open job requires). A plan is a set $T \subseteq C$ of k workshops; a workshop raises every attendee to at least Intermediate in its skill. Writing $n_s(T)$ for the number of jobs within reach of s after the plan, the two objectives are

$$f_{\text{reach}}(T) = |\{\, s : n_s(T) > 0 \,\}|, \qquad f_{\text{match}}(T) = \Sigma_s\, n_s(T) \tag{5.6}$$

The goal *Reach every student* maximises (f_{reach}, f_{match}) in that order; *Most matches overall* maximises (f_{match}, f_{reach}). The placement cell chooses the goal, and the tool shows the trade-off rather than hiding it in a weight.

5.6.2 Why Greedy Selection Fails

The objective is not submodular. Take one student with skills {Git} and one job requiring {Java, Spring Boot, Git}. Then f({Java}) = f({Spring Boot}) = f(∅) = 0, because 2 of 3 skills is 67%, while f({Java, Spring Boot}) = 1. The value of Spring Boot grows once Java is chosen, the opposite of diminishing returns. A greedy planner sees zero gain for each skill alone and may never choose either, so the $1 - 1/e$ guarantee [5] does not hold.

5.6.3 Exact Search with Bit Masks

Candidate skills are indexed and every student and job becomes a set of bit masks (one per level and weight class), so testing (5.3) for a student and a job costs a few OR, AND and population-count operations. The planner enumerates all $C(|C|, k)$ subsets, scores each in $O(|S| \cdot |J|)$, and keeps the best for the chosen goal. If $C(|C|, k) \cdot |S| \cdot |J|$ would exceed 3×10^8, it falls back to greedy selection and says so on screen. For each student it then finds the smallest subset of the plan that gives the same result, and invites the student only to those workshops; a workshop nobody needs is dropped. Figure 5.1 summarises the algorithm.

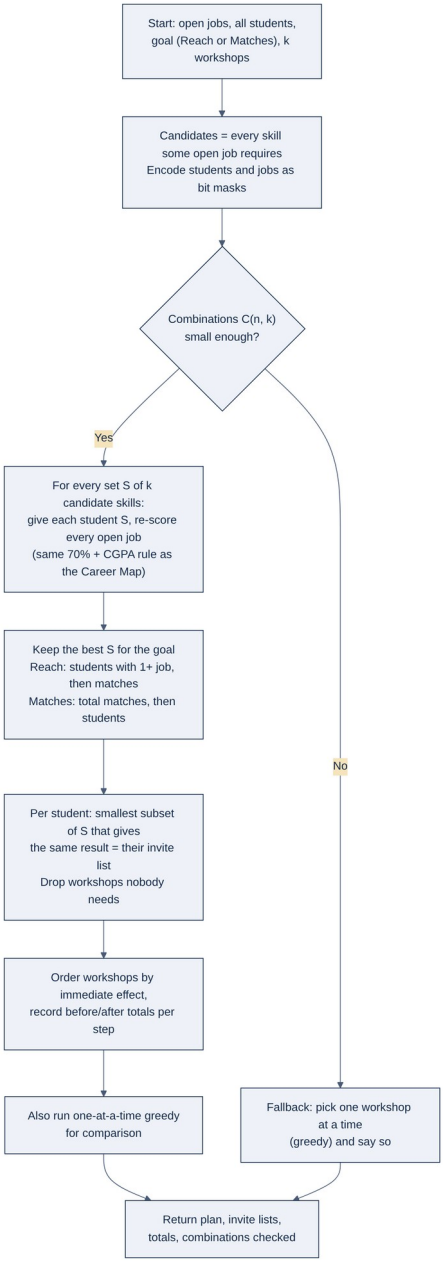


Figure 5.1: Flow of the exact Workshop Planner

5.7 Next Best Action

The Career Map shows one suggested action chosen by fixed priority rules: complete the profile; upload a résumé; give placement consent; add skills; apply to a job already within reach; otherwise learn the top Opportunity Math skill. Each suggestion carries a reason and a link to the page where it can be done.

5.8 Résumé Score and Readiness Points

The résumé score out of 100 is computed from ten fixed rules (Table 5.1), so the same profile always gets the same score, and every missing point is shown with how to earn it.

Readiness points for the leaderboard add 15 per verified skill, 10 per verified certificate and 5 per job within reach to the résumé score.

Table 5.1: Résumé score rules

Rule	Points	Maximum
Résumé uploaded	15	15
About you: 150+ characters (5 points for 50+)	10	10
Skills listed, up to 5	4 each	20
Skills at Intermediate or above, up to 5	2 each	10
Skills verified by assessment or certificate, up to 3	5 each	15
Verified certificates, up to 2	5 each	10
GitHub link	5	5
LinkedIn link	5	5
Introduction video	5	5
CGPA, 10th and 12th marks filled in	5	5
Total		**100**

5.9 Skill Assessment Grading

An assessment draws 8 questions for one of 15 skills, 4 basic and 4 advanced, from a bank of 120 questions. The options of each question are shuffled and the order is stored with the attempt; the browser receives the questions and options but never the answers. The student has 10 minutes (with a 90-second grace for network delay), and grading happens on the server. Eight correct answers verify Advanced, six or seven Intermediate, four or five Beginner; with fewer than four nothing is verified and the student may try again after 24 hours. A verified level replaces the self-reported one in (5.2) and is locked against editing.

5.10 Career Copilot

The Copilot uses Gemini function calling [16] with eight tools: the student’s snapshot, job matches, skill opportunities, a learning simulation, a job explanation, application status, and two action proposals (apply to a job; show a skill on the map). The server runs each tool for the student identified by the token, so the model cannot read another student’s data. Proposed actions appear as buttons and run only when the student clicks them, and every answer lists the data it used. If there is no key, or the API fails after retries and a fallback to a lighter model, an offline mode answers the most common questions with the same tools.

5.11 Evidence-Checked Résumé Skill Finder

Text is extracted from PDF with Apache PDFBox and from Word files with Apache POI. Gemini, in JSON mode, returns candidate skills, each with a quoted evidence line. The server normalises case, spacing and punctuation and keeps a suggestion only if the quote occurs in the résumé text and the skill maps to the catalogue, so an invented skill cannot reach the student. Without a key, word-boundary patterns with explicit disambiguation are used: JavaScript is not Java, “react quickly” is not React, and a grade of C is not the C language.

CHAPTER 6
IMPLEMENTATION

6.1 Technology Stack

Table 6.1: Technology stack

Layer	Technology	Use in the project
Frontend	React 18, React Router 6, Vite 5	All screens, routing by role, development server and build
	Axios, SVG, CSS	API calls with the token; Career Map drawing; design tokens
Backend	Java 17, Spring Boot 3.3	REST API, dependency injection, configuration
	Spring Security, jjwt, BCrypt	Authentication and role rules
	Spring Data JPA, Hibernate, Bean Validation	Database access and request validation
Database	MySQL 8; H2 for tests	Persistent data; in-memory database for tests
AI	Google Gemini API	Copilot (function calling) and résumé skill finder (JSON mode)
	Apache PDFBox, Apache POI	Text extraction from PDF and Word résumés
Clients	SwiftUI (Xcode), Kotlin (Android Studio)	Mac, iPhone and Android apps around the web app
Tooling	Maven, npm, VS Code tasks	Build, test, run and debug
Testing	JUnit 5, MockMvc	Unit and end-to-end API tests
Deployment	Docker, Docker Compose, Railway	One image with the web app inside the backend; online hosting

Figure 6.1 is a mind map of the whole project: what each user can do, the algorithms, and every technology, with where each lives in the code.

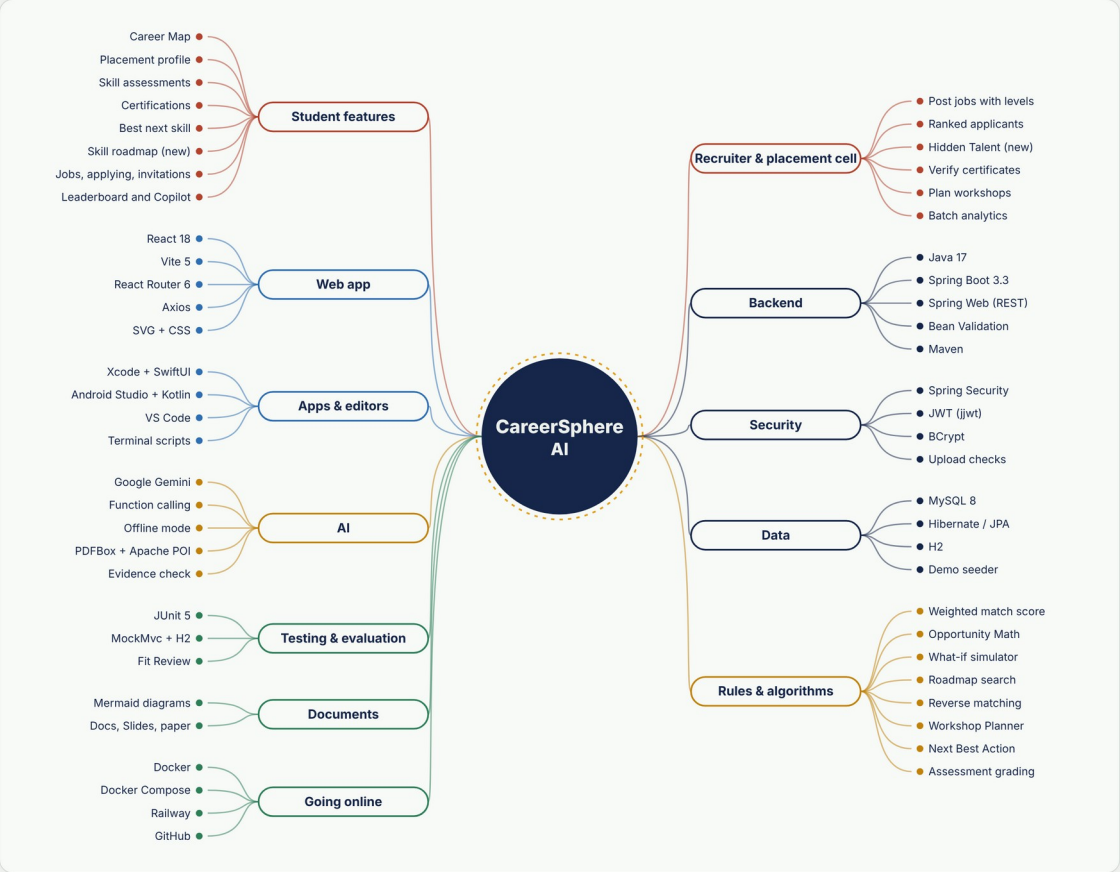


Figure 6.1: Mind map of the project: features, algorithms and technologies

6.2 Project Structure

The repository has one folder per part. The backend has 68 Java source files (about 6,400 lines) and 8 test files (about 1,270 lines); the frontend has 43 JavaScript and JSX files (about 3,900 lines) and one stylesheet; the Android app is about 270 lines of Kotlin and the Mac and iPhone app about 300 lines of Swift.

Listing 6.1: Folder structure

```
careersphere/
├── backend/            Spring Boot API (Java 17, Maven)
│   └── src/main/java/com/careersphere/
│       ├── ai/          Copilot, Gemini client, résumé skill finder
│       ├── config/      security, uploads, demo data seeder
│       ├── controller/  REST endpoints per role
│       ├── dto/         request and response records
│       ├── entity/      JPA entities
│       ├── repository/  Spring Data repositories
│       ├── security/    JWT filter and service
│       └── service/     match rule, roadmap, talent, workshops, ...
├── frontend/           React + Vite web app
│   └── src/  pages/ (student, recruiter, admin), components/, services/
├── android/            Android Studio project (Kotlin WebView)
├── app/                Xcode project (SwiftUI, Mac and iPhone)
├── .vscode/            tasks and debug configurations
├── docs/               diagrams, evaluation kit, brand
├── Dockerfile, docker-compose.yml, deploy/
└── run.sh, stop.sh     one-command start and stop on a Mac
```

6.3 Backend Implementation

The backend follows the usual Spring layering. Controllers map URLs to service calls and convert results to DTO records. Services hold all rules and run inside transactions. Entities map to tables, with skill levels stored as element collections keyed by skill. A global exception handler turns errors into a consistent JSON shape with a message and field errors. The core of the match rule is only a few lines (Listing 6.2).

Listing 6.2: Match score (MatchService.java)

```
public static int weight(Job job, Skill skill) {
    return job.isNiceToHave(skill) ? 1 : 2;
}
/** Credit in half units: 2w if level >= needed, w if 0 < level < needed, 0 i
f missing. */
public static int halfCredit(int level, int needed, int weight) {
    if (level <= 0) return 0;
    return level >= needed ? 2 * weight : weight;
}
/** round(100 x half / total), half up, in integers. */
public static int percent(long half, long total) {
    if (total <= 0) return 100;
    return (int) ((200L * half + total) / (2L * total));
}
public int score(Map<Long, Integer> levels, Job job) {
    long half = 0, total = 0;
    for (Skill s : job.getSkills()) {
        int w = weight(job, s);
        total += 2L * w;
        half += halfCredit(levels.getOrDefault(s.getId(), 0), job.minLevelOf(
s), w);
    }
    return percent(half, total);
}
public boolean reachable(Map<Long, Integer> levels, Double cgpa, Job job) {
    return cgpaEligible(cgpa, job) && score(levels, job) >= MATCH_THRESHOLD;
// 70
}
```

The Skill Roadmap search is equally direct: it walks every combination of *k* actions with at most one per skill and keeps the best (Listing 6.3).

Listing 6.3: Bundle search (RoadmapService.java, simplified)

```
for (int k = 1; k <= Math.min(MAX_STEPS, distinctSkills(actions)); k++) {
    List<Action> best = null;
    int bestReach = -1, bestEffort = Integer.MAX_VALUE;
    for (List<Action> combo : combinations(actions, k)) {
        checked++;
        int r = reach(apply(levels, combo), cgpa, open);
        int effort = combo.stream().mapToInt(Action::effort).sum();
        if (r > bestReach || (r == bestReach && effort < bestEffort)) {
            best = combo; bestReach = r; bestEffort = effort;
        }
    }
    // keep looking: a size can gain nothing while the next size gains a lot
    if (best == null || bestReach <= lastReach) continue;
    bundles.add(describe(levels, cgpa, open, before, now, best, single));
    lastReach = bestReach;
}
```

6.4 Frontend Implementation

The React application has one page component per screen, grouped by role, and shared components for the layout, skill input with level chips, match pills and explanations. A small service layer wraps every API call; the Axios instance adds the token, turns server errors into readable sentences and sends the user to the login page when the token expires. The Career Map is drawn in SVG, with each job's distance from the centre equal to 100 minus its score, and animated when skill levers change. Routes are protected by role, so a student cannot open recruiter pages even by typing the address.

6.5 Screens

Figures 6.2 to 6.20 show the main screens, taken from the running application with the demo data.

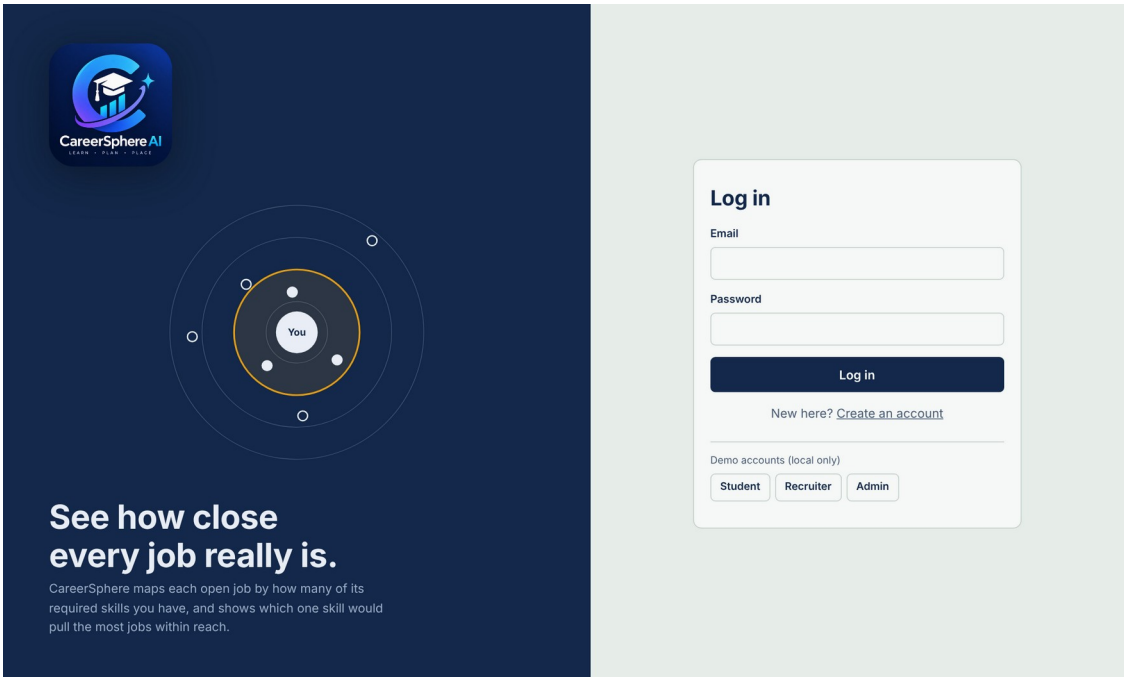


Figure 6.2: Login page with demo accounts for the three roles

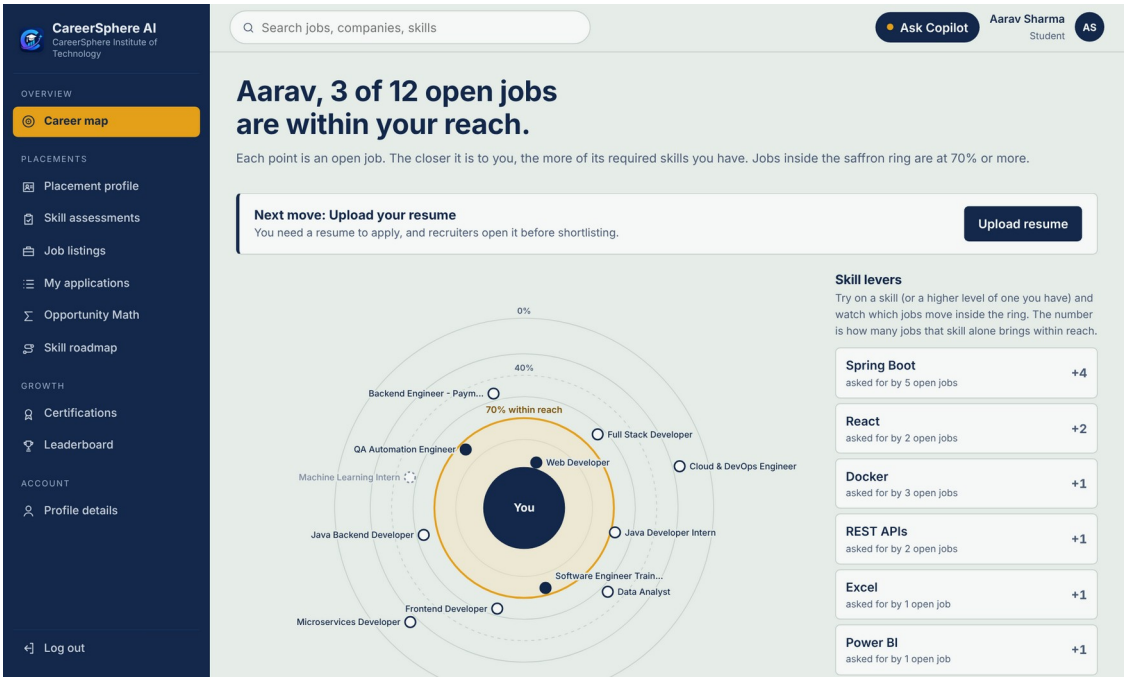


Figure 6.3: Career Map: each open job placed by match score; skill levers on the right

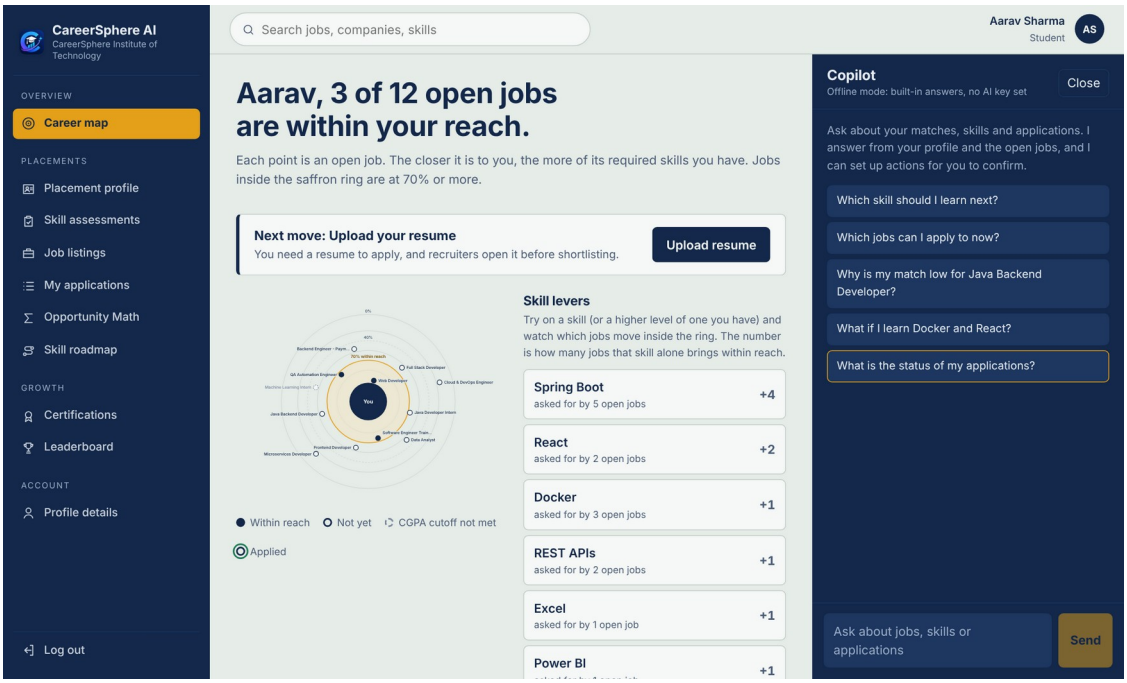


Figure 6.4: Career Map with the Copilot panel open

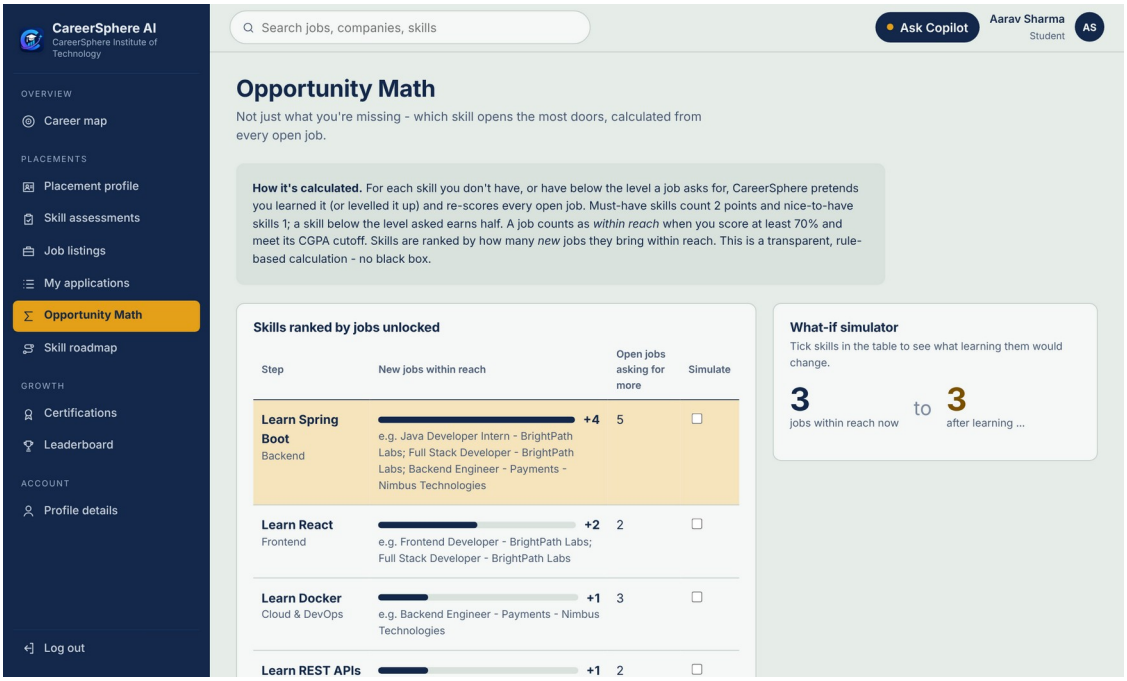


Figure 6.5: Opportunity Math: the skills that bring the most jobs within reach

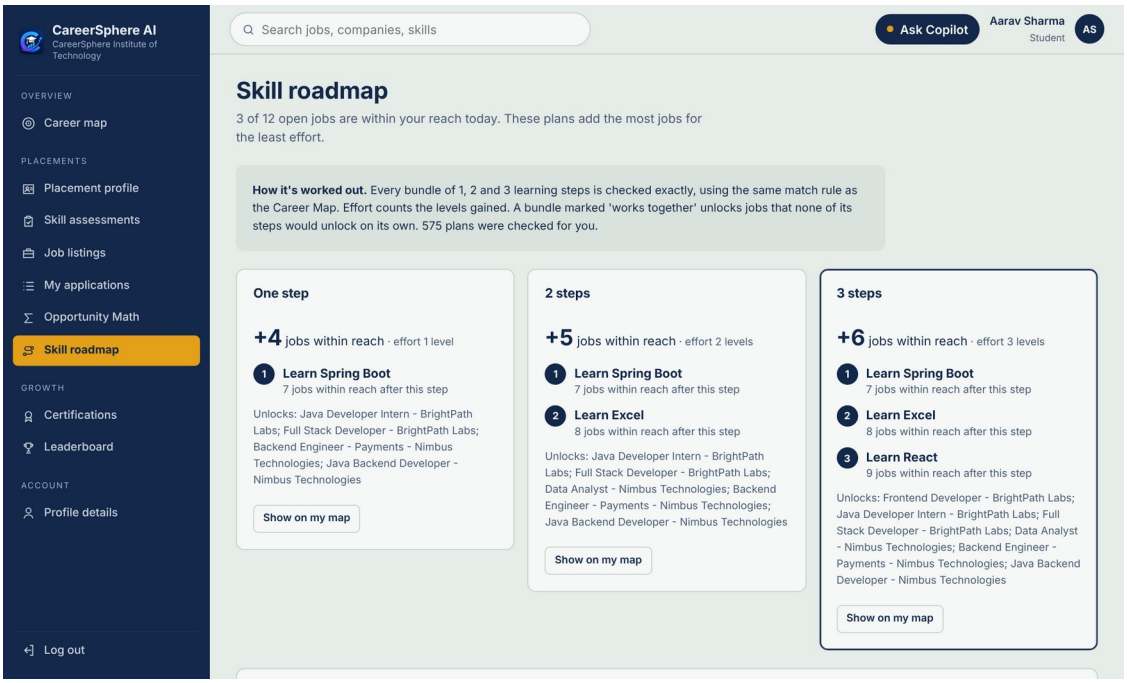


Figure 6.6: Skill Roadmap: best one-, two- and three-step plans

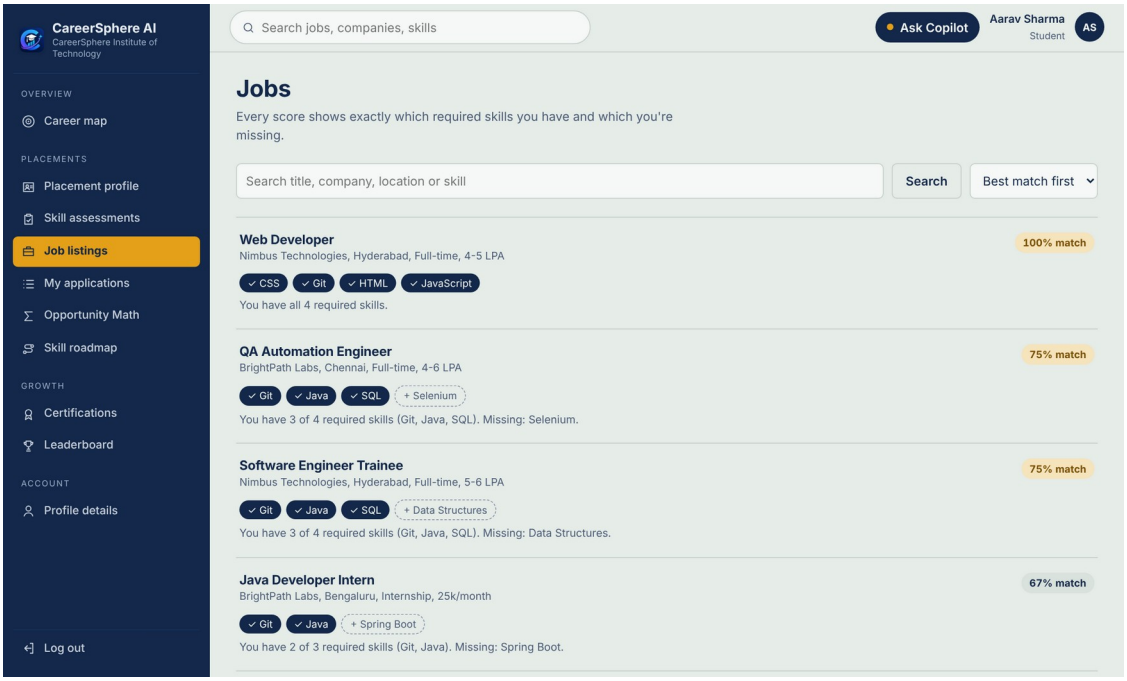


Figure 6.7: Job listings with explained match scores

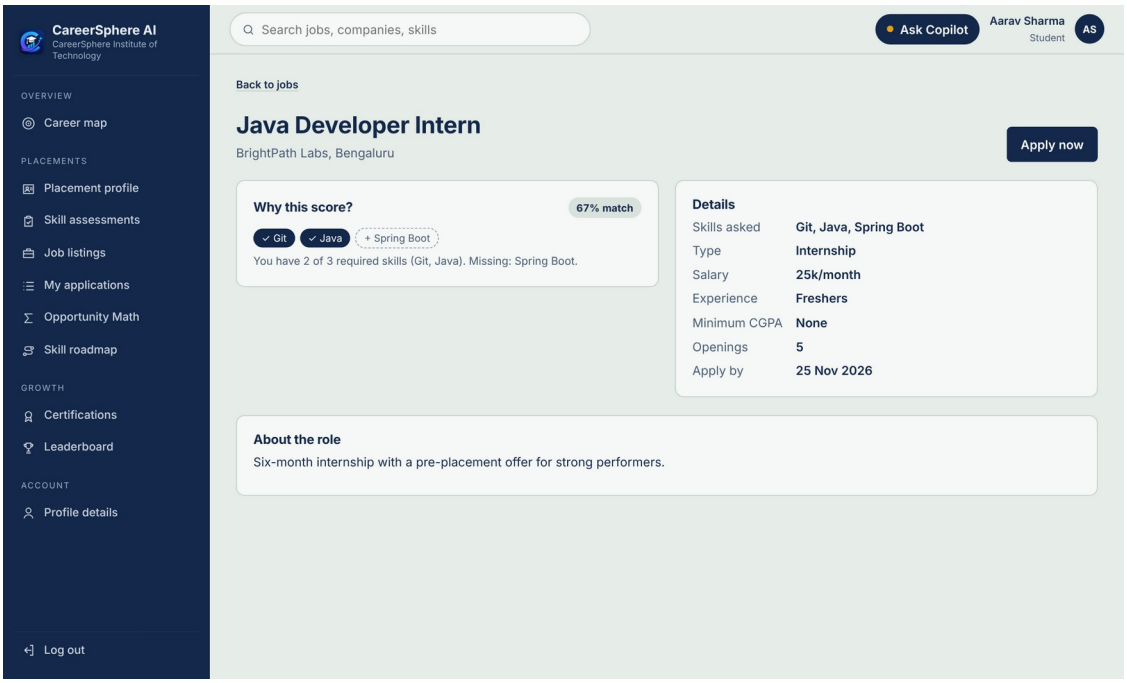


Figure 6.8: Job detail: why this score, and the job's requirements

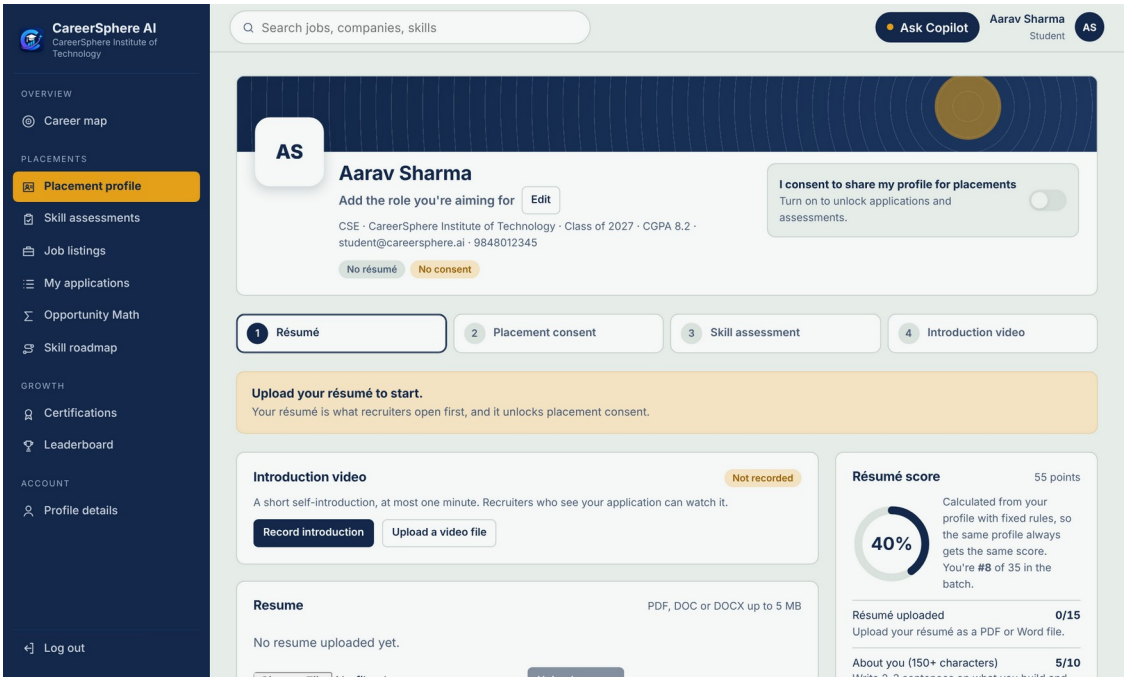


Figure 6.9: Placement profile: guided steps and résumé score (new student)

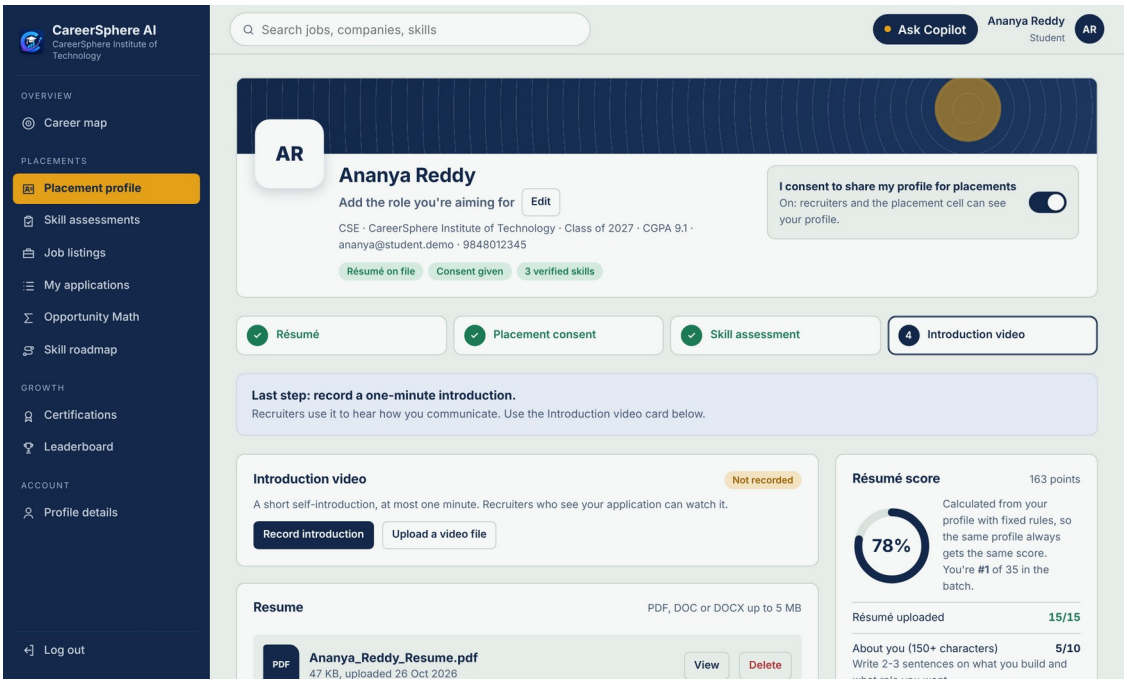


Figure 6.10: Placement profile of a student who has completed most steps

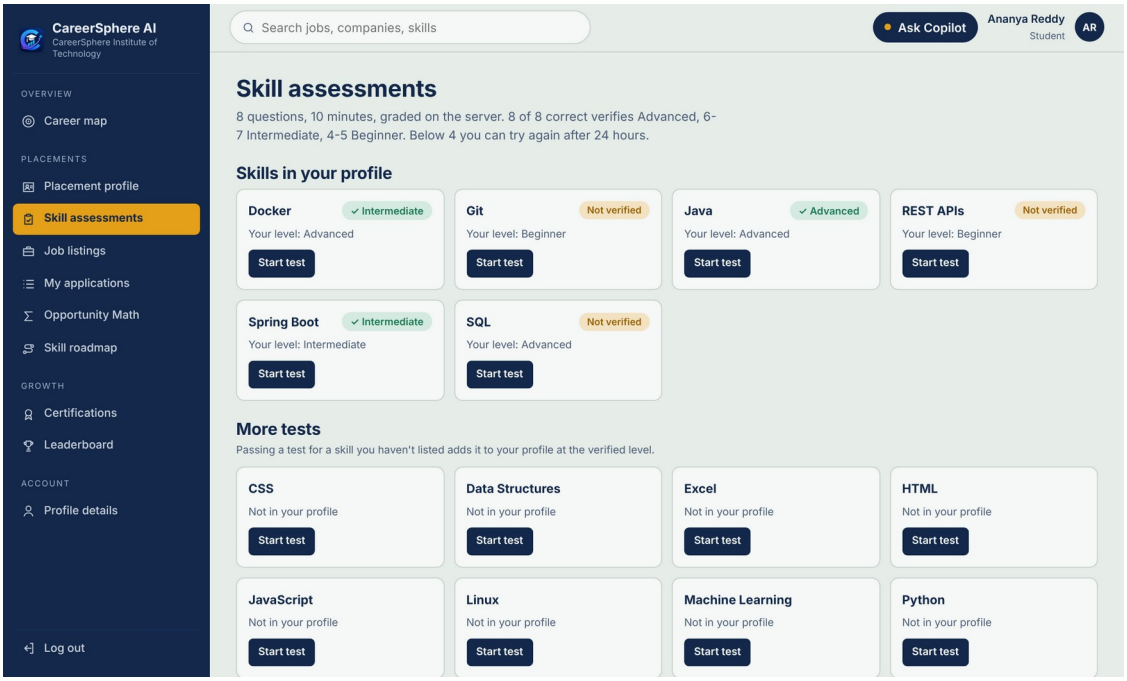


Figure 6.11: Skill assessments: verified and unverified skills

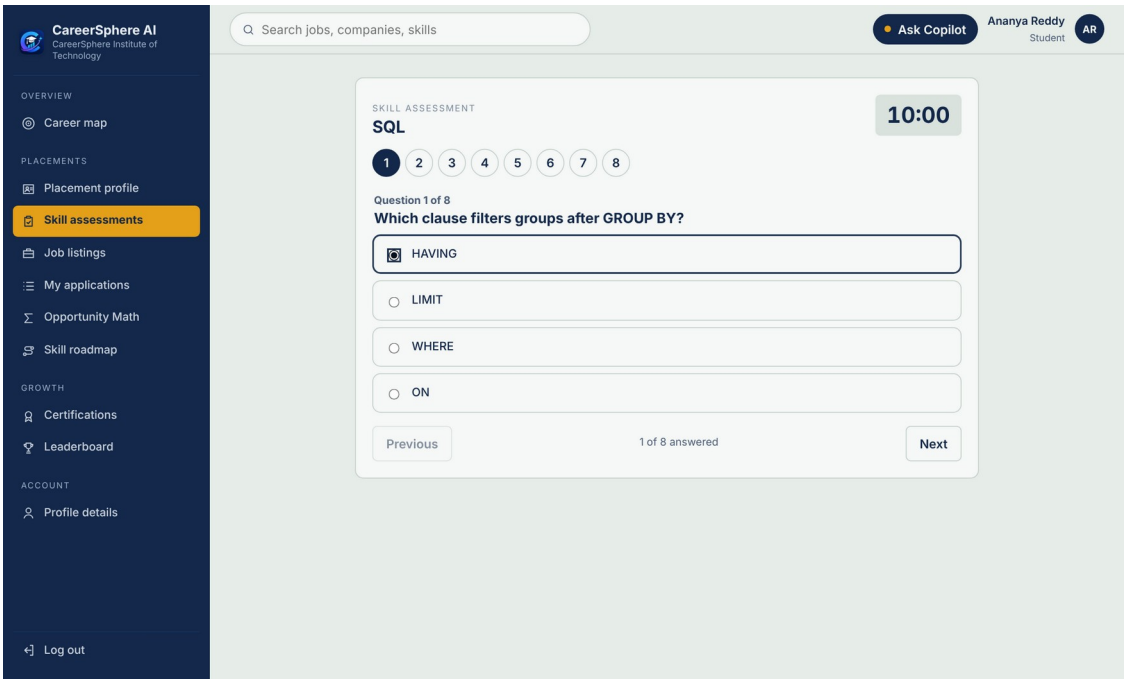


Figure 6.12: A timed assessment in progress

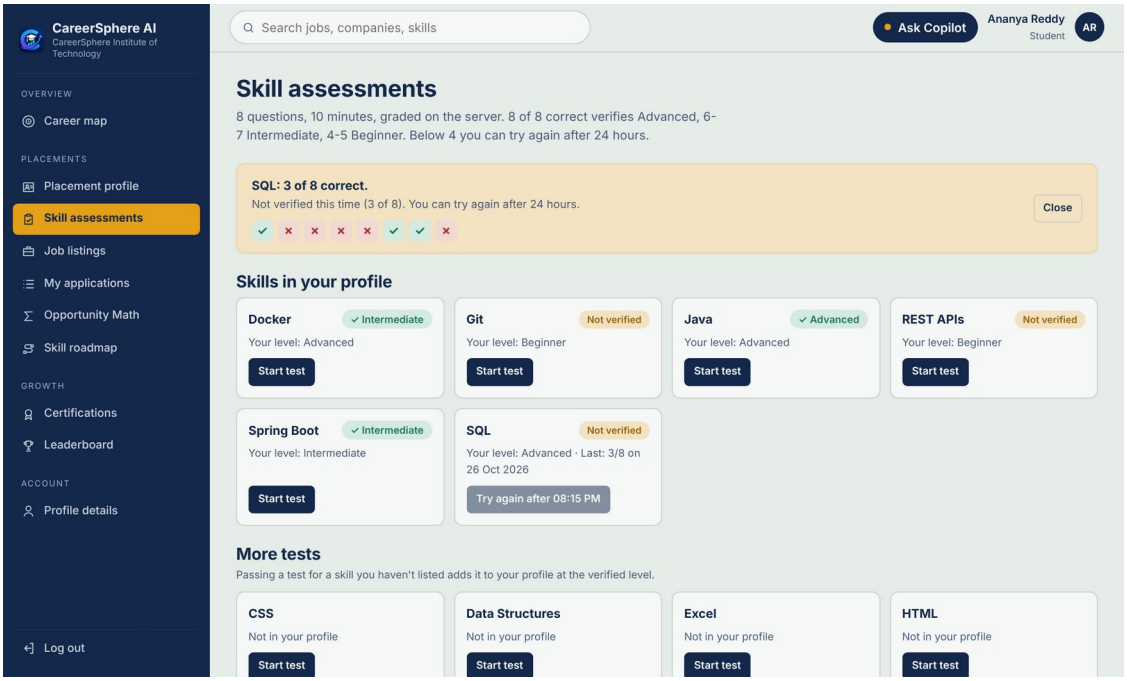


Figure 6.13: Assessment result: fewer than 4 correct verifies nothing, and a retake is allowed after 24 hours

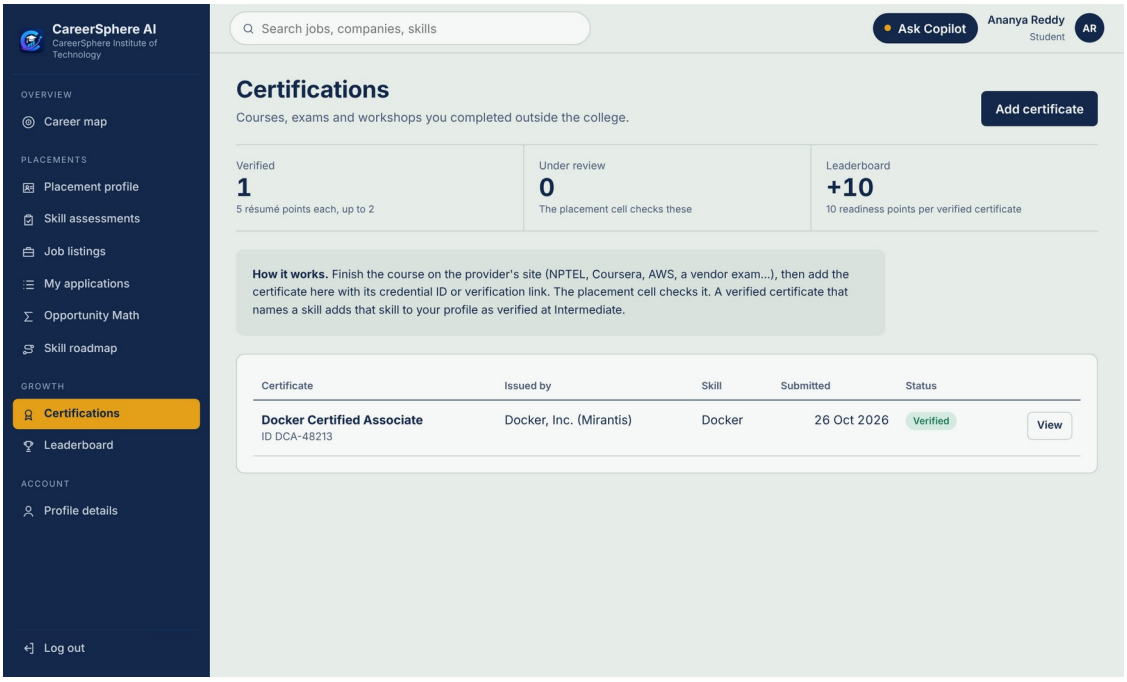


Figure 6.14: Certifications: a verified certificate and how verification works

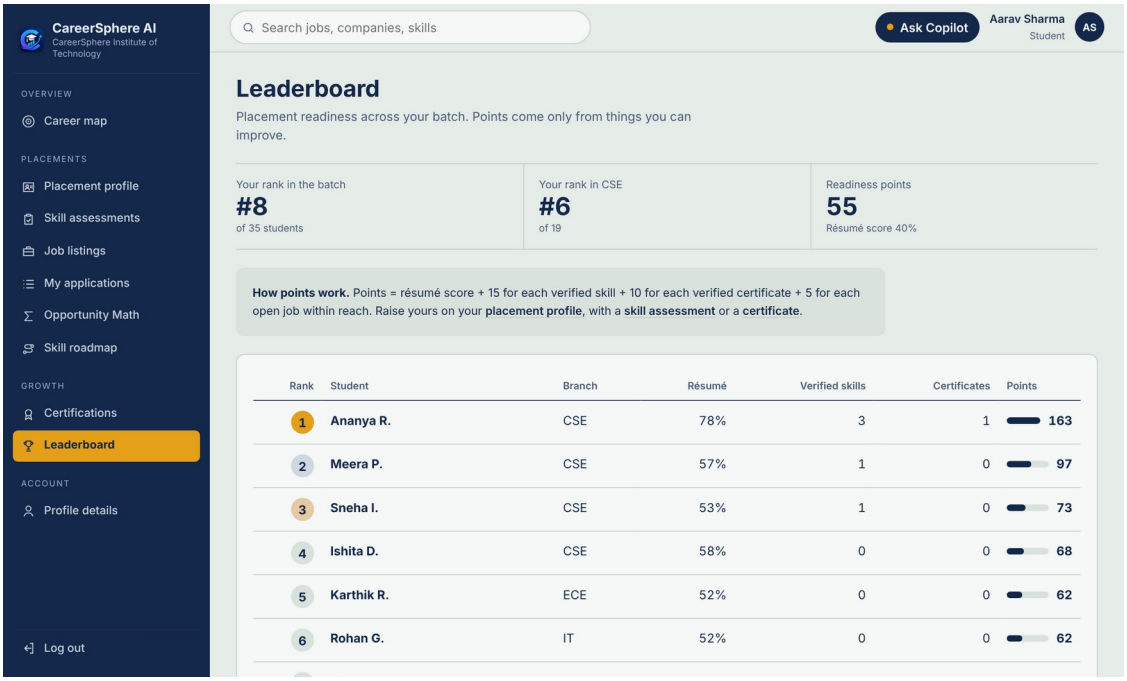


Figure 6.15: Readiness leaderboard

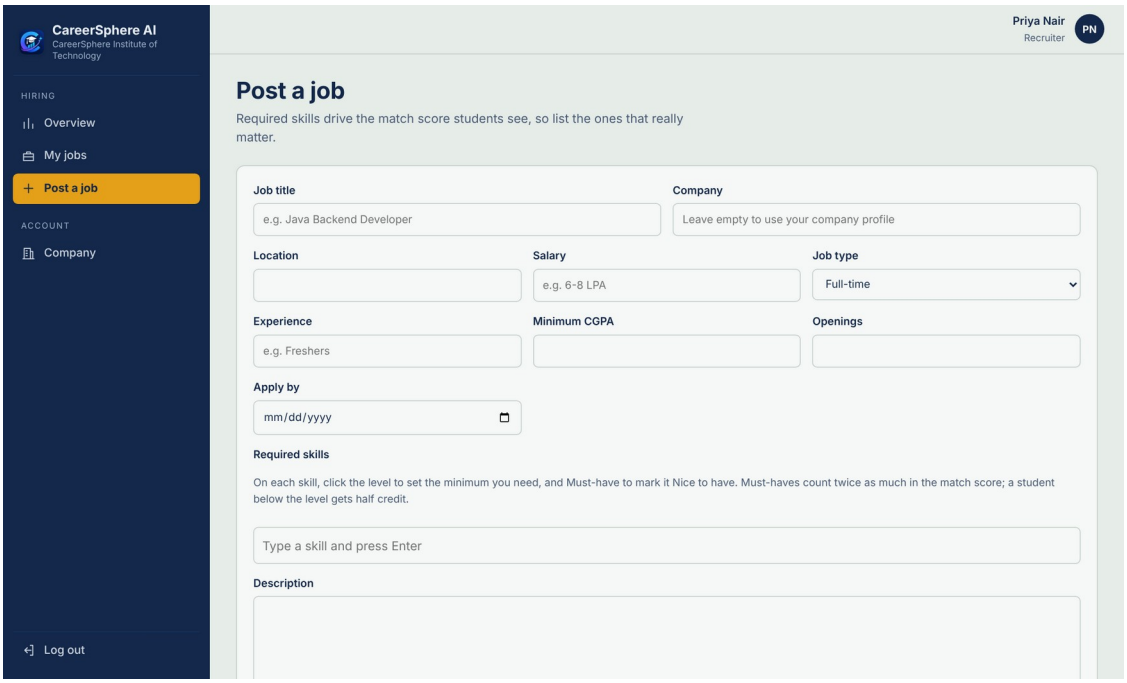


Figure 6.16: Recruiter: posting a job with skill levels and must-have or nice-to-have flags

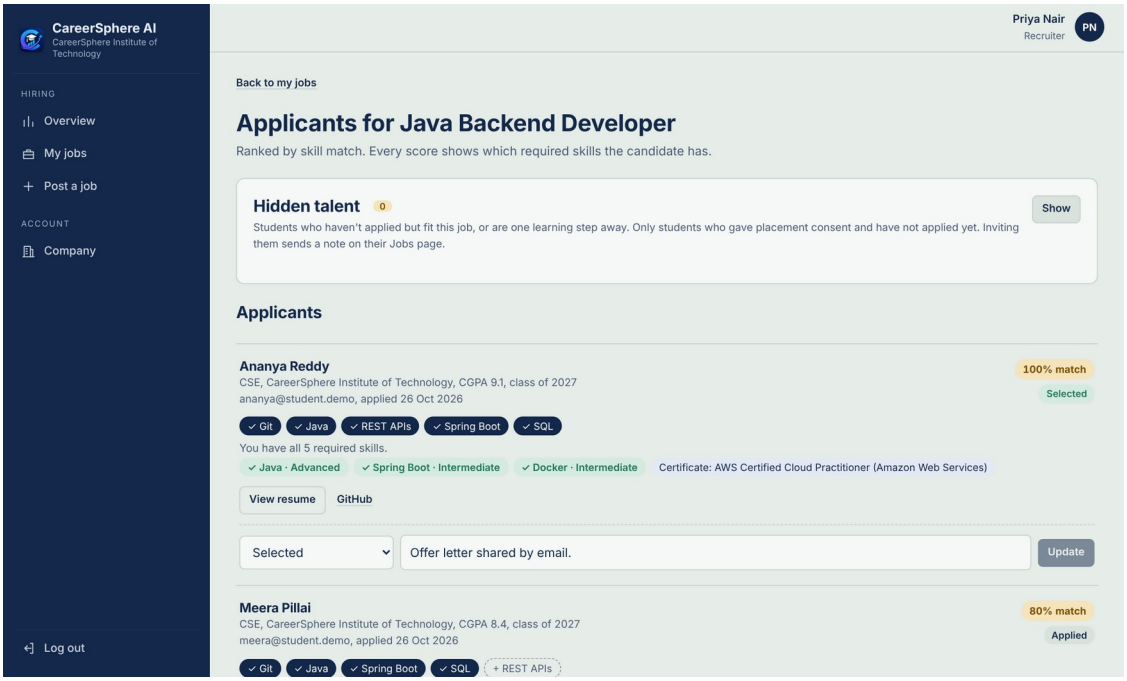


Figure 6.17: Recruiter: applicants ranked by match, with verified skills

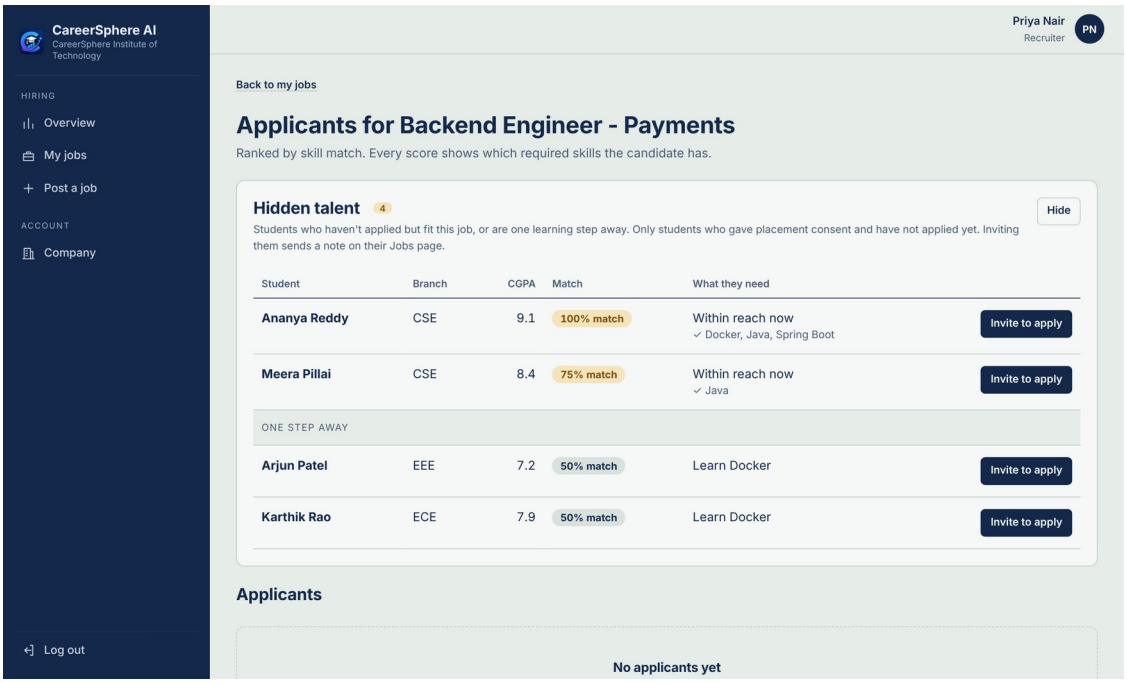


Figure 6.18: Recruiter: Hidden Talent for a job, with invitations

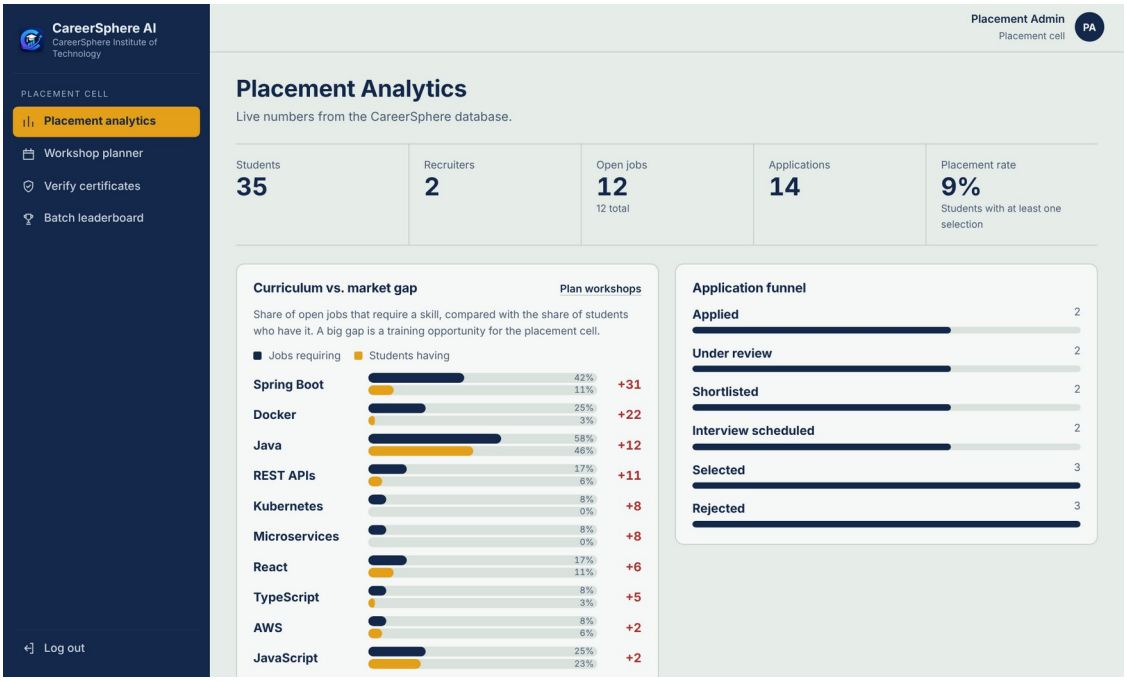


Figure 6.19: Placement cell: analytics and the curriculum–market skill gap

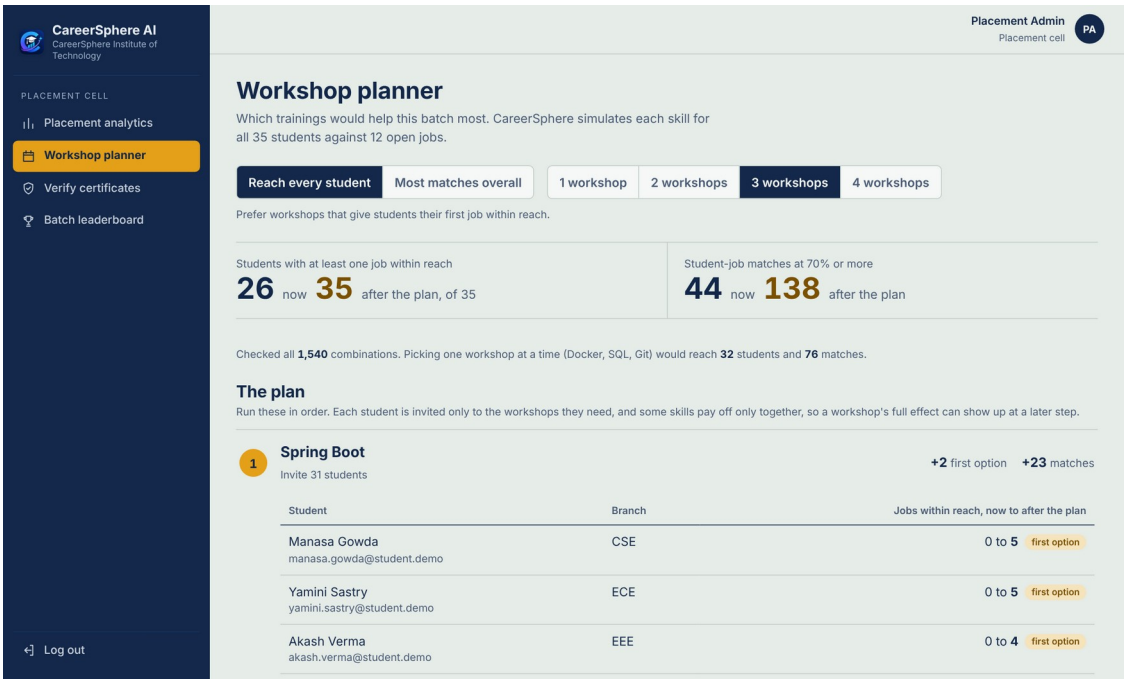


Figure 6.20: Placement cell: Workshop Planner for three workshops

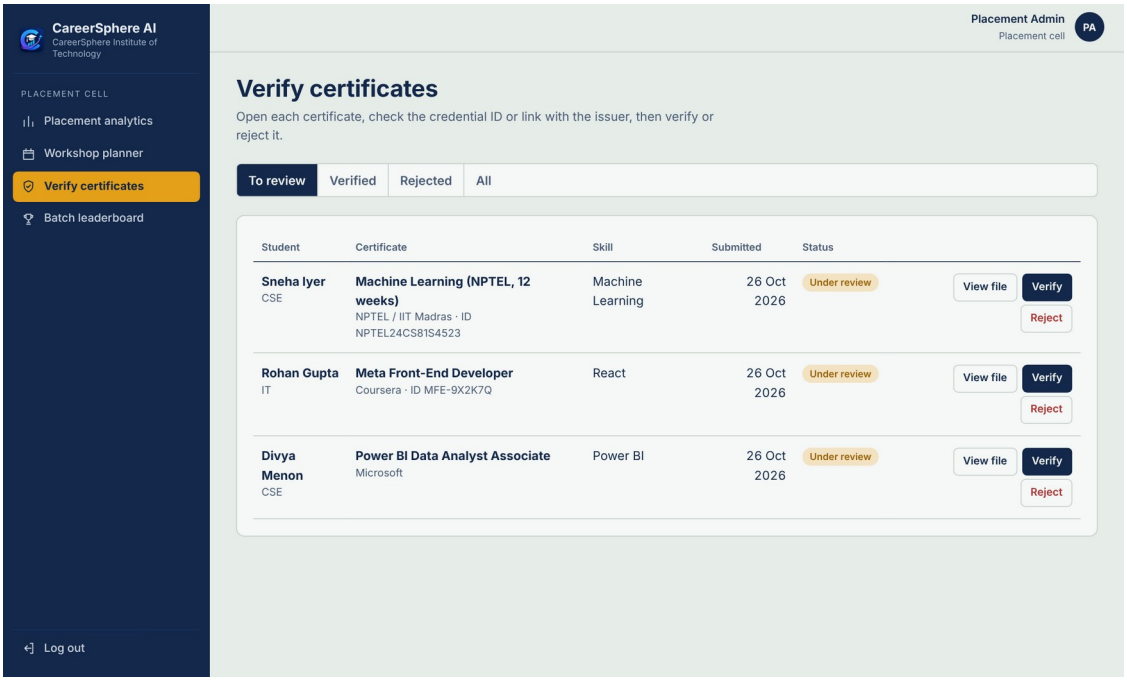


Figure 6.21: Placement cell: certificates waiting for verification

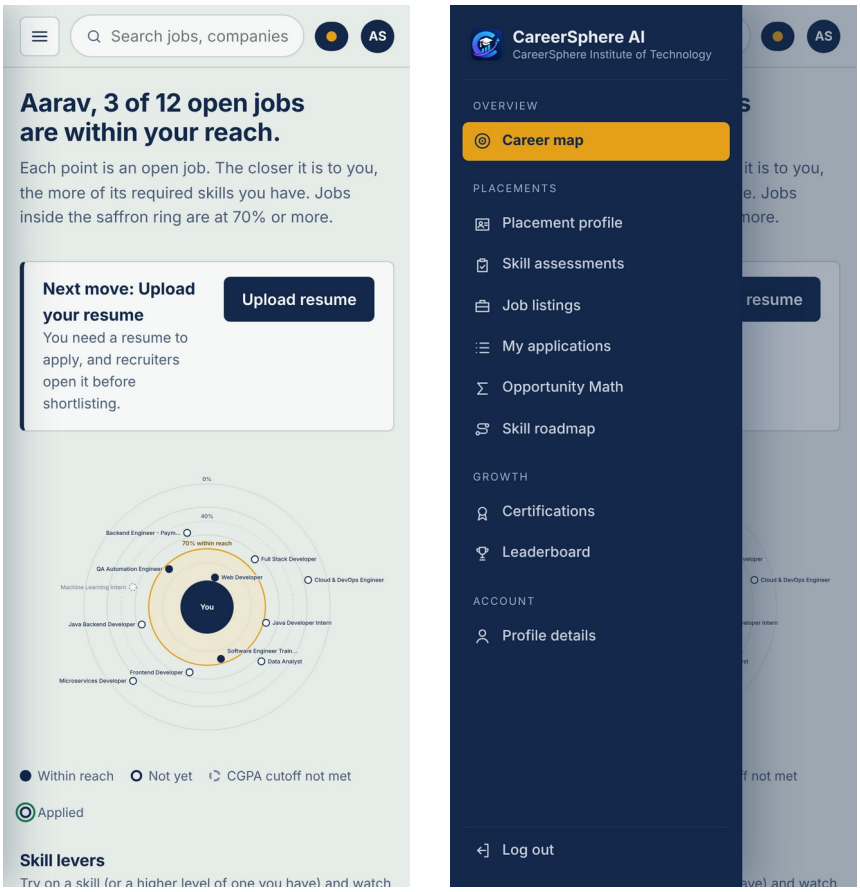


Figure 6.22: The same app on a phone: Career Map and the slide-in menu

6.6 Running on Different Platforms

- **Terminal (Mac):** `./run.sh` starts MySQL, the backend and the web app, each in its own window, and opens the browser; `./stop.sh` stops them.
- **Xcode:** the CareerSphere project builds a SwiftUI app for Mac and iPhone that shows the web app in a WKWebView; a build step starts the servers. Camera and microphone permissions allow recording the introduction video.
- **VS Code:** tasks start, test and stop the system, and launch configurations debug the Java backend and the React app with breakpoints.
- **Android Studio:** a Kotlin app shows the web app in a WebView with file upload (including recording a video with the camera), a résumé viewer, pull to refresh and a setting for the server address.
- **Online:** a single Docker image builds the web app and puts it inside the backend jar; Docker Compose runs it with MySQL on any computer, and Railway hosts it with a public HTTPS address (Appendix A).

CHAPTER 7
TESTING

7.1 Testing Strategy

The system is tested at three levels. **Unit tests** check the rules in isolation: the match score, Opportunity Math, the Skill Roadmap, Hidden Talent, the Workshop Planner, the résumé score, assessment grading and the résumé skill patterns. They build small, hand-checkable cases in memory, so every expected value can be worked out on paper. **API tests** start the whole application on an in-memory H2 database and call the real endpoints through the real security filter chain with MockMvc, covering each feature from registration to invitations. **User-interface checks** walk through every screen in a real browser at desktop and phone widths and fail on any script error.

7.2 Unit Tests

There are 39 unit tests in six classes (Table 7.1), all passing.

Table 7.1: Unit tests

Class	Tests	What they check
MatchServiceTest	9	Share of skills; rounding (2 of 3 = 67%); 100% case; CGPA cutoff reported separately; missing CGPA; inclusive 70% threshold; Opportunity Math ranking, CGPA exclusion and no known skills suggested
SkillLevelTest	6	Defaults equal the plain rule; half credit below the level; nice-to-have weight; level-up suggestions; workshop to Intermediate; what-if “Skill:2” syntax
WorkshopServiceTest	6	Baseline counts; first-reach ordering; three workshops reach all students; unneeded workshops dropped; no invitation for a skill already known; a complementary case where greedy fails
RoadmapTalentTest	4	Two-step bundle when no single skill helps, marked “works together”; least effort wins ties; cheapest path to a job and CGPA blocker; Hidden Talent excludes non-consented students and applicants
PlacementProfileTest	8	Résumé score adds up and never passes 100; steps in order; leaderboard privacy; grading levels; lock before consent; answers never sent; retake wait; resuming a running test
ResumeSkillServiceTest	6	Skills in project lines with evidence; common spellings; JavaScript is not Java; ordinary words not mistaken for skills; known skills marked; long evidence trimmed

As a check that the tests can detect a faulty rule, the threshold comparison was changed from ≥ to >; the inclusive-threshold test failed at once, and passed again when the change was reverted.

7.3 API Test Cases

Table 7.2 lists the 17 end-to-end test cases. Each runs against a fresh in-memory database with the demo data, through the same security configuration as production. They are run with `mvn test` from the backend folder.

Table 7.2: End-to-end API test cases

ID	Test case	Expected result
TC-01	Register; register again with the same email	Token returned; second attempt rejected (409)
TC-02	Register with role ADMIN	Rejected (403)
TC-03	Log in with a wrong password	401 with a clear message
TC-04	Call a protected API without or with a bad token	401
TC-05	Student calls recruiter and admin APIs, and so on	403 for every other role’s API
TC-06	Save profile with CGPA 11.5; save a valid profile	400 with a field error for CGPA; valid profile saved and reloaded
TC-07	Upload a real PDF; upload other files (e.g. an .exe)	PDF accepted; other files rejected (400)
TC-08	Run the résumé skill finder; confirm some skills	Suggestions with evidence; only confirmed skills added
TC-09	Apply without résumé, without consent, then twice	Blocked (400), blocked (400), accepted once, then 409
TC-10	Another recruiter updates an application	403; the owner’s update is visible to the student
TC-11	What-if with extra skills	Jobs within reach never decrease
TC-12	Ask the Copilot without an AI key	Offline answer built from the student’s data
TC-13	Admin requests a workshop plan; student requests it	Plan returned; student gets 403
TC-14	Post a job with levels and nice-to-have skills	Saved and used in the score
TC-15	Consent without résumé; assessment before and after consent	Consent blocked; test locked, then graded on the server
TC-16	Upload a certificate; admin verifies it	Skill added to the profile as verified
TC-17	Student roadmap; recruiter talent and invite	Roadmap returned; candidate listed; invitation shown to the student; repeat invite 409

7.4 User Interface Testing

A scripted browser (Playwright with Chromium) logs in as each role and opens every screen in turn: Career Map, Opportunity Math, Skill Roadmap, jobs, job detail, placement profile, assessments (including taking and submitting a test), certifications, leaderboard, recruiter jobs, job form, applicants, Hidden Talent (including sending an invitation and checking that it appears on the student's Jobs page), admin analytics, Workshop Planner and certificate review. The run fails on any JavaScript error. The same walk is repeated at a phone width of 390 pixels, where the check also confirms that no page scrolls sideways. The screenshots in Chapter 6 come from this walk.

7.5 Validation and Error Handling

Every form is validated on the server as well as in the browser. Invalid input returns status 400 with a message for each field, for example "CGPA must be between 0 and 10". Business-rule violations return a plain sentence, for example "Give placement consent on your placement profile before applying, so recruiters may see your profile." Unexpected errors are logged on the server and return a generic message, so no stack trace reaches the browser.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Demo Data

All results use the fictional demo batch shipped with the system: 35 students from four branches with 3.49 skills on average, and 12 open jobs from two fictional companies requiring 4.33 skills on average, 4 of them with a CGPA cutoff. There are 22 candidate skills and 420 student–job pairs. Scores are 0% for 144 pairs, 1–39% for 141, 40–69% for 91, 70–99% for 30 and 100% for 14. Today 26 students have at least one job within reach, with 44 matches in total. The demo data was designed to resemble a mid-sized department; the results show how the methods behave, not their effect on real placements.

8.2 Which Skill to Learn Next

We compare the top Opportunity Math action with the *demand* heuristic, the missing skill required by the most open jobs, which is what a skill-gap list usually shows first. The measure is how many jobs the recommended skill brings within reach (Table 8.1).

Table 8.1: Recommended next skill, 35 students

Measure	Demand heuristic	Opportunity Math
Mean jobs brought within reach	1.03	**1.43**
Total jobs brought within reach	36	**50**
Students whose recommendation unlocks nothing	20	**7**
Students with a different recommendation	–	13

Opportunity Math is, by definition, the best single action under (5.4); the table shows how far the usual heuristic falls short. For 20 of 35 students the most-demanded missing skill would not bring a single job within reach. For 7 students no single skill helps at all, because every job is two or more skills away.

8.3 Skill Roadmap

Table 8.2 shows what happens when the student may take two or three steps.

Table 8.2: Skill Roadmap, 35 students

Steps allowed	Mean new jobs within reach	Students helped
1 (Opportunity Math)	1.43	28
up to 2	2.89	**35**
up to 3	**4.71**	**35**

All 7 students for whom no single skill helps gain at least one job with two steps. One student, for example, gains nothing from any one skill, but three jobs from learning Java and SQL together. Of the 65 multi-step bundles shown across the batch, 59 are marked *works together*: on this batch the value of a second or third step comes mostly from combining skills, which is exactly what one-at-a-time advice cannot see. The search checks 1,083 bundles per student on average and runs on every page load.

8.4 Workshop Planning for the Batch

Exact search is compared with three baselines for k = 1 to 4 workshops: *greedy*, which repeatedly picks the workshop with the largest immediate effect under the same goal; *top-k demand*, the k skills required by the most open jobs; and *top-k gap*, the k skills with the largest gap between job demand and student supply, as shown on the admin dashboard. For the two top-k baselines every student lacking a chosen skill attends.

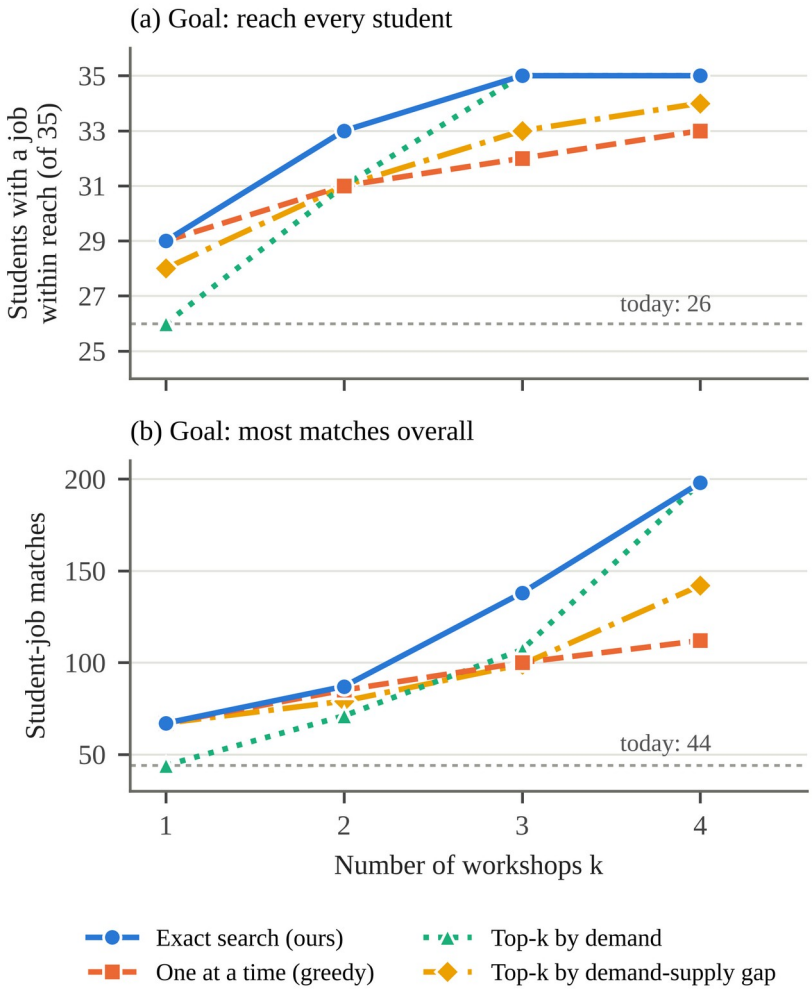


Figure 8.1: Outcome of each planning method for k = 1 to 4 workshops. Dotted line: today. (a) Reach goal; (b) Matches goal.

Table 8.3: Workshop plans: students reached / matches (Reach goal)

k	Exact	Greedy	Top-k demand	Top-k gap
1	**29** / 56	29 / 56	26 / 44	28 / 67
2	**33** / 65	31 / 64	31 / 71	31 / 79
3	**35** / **138**	32 / 76	35 / 107	33 / 99
4	**35** / **198**	33 / 81	35 / **198**	34 / 142

With three workshops, exact search selects Spring Boot, Java and Git and gives every student a job within reach, with 138 matches; greedy selects Docker, SQL and Git and reaches 32 students with 76 matches. Under the *Most matches overall* goal greedy reaches 100 matches against 138. Greedy falls behind even the simple top-k demand baseline from k = 3, which illustrates Section 5.7.2: Java helps few students on its own and pays off only together with Spring Boot and Git. At k = 2 the two goals disagree: *Reach* chooses Data Structures and Java (33 students, 65 matches), while *Matches*

chooses Spring Boot and Git (29 students, 87 matches). The three-workshop plan invites 31, 19 and 18 students, each only to the workshops they need.

Table 8.4: Planner runtime, median of 9 calls

k	Combinations	Time (ms)
1	22	0.9
2	231	1.9
3	1,540	6.5
4	7,315	29.3
5	26,334	81.7
6	74,613	228.9

Table 8.4 shows that exact search is interactive for the workshop counts a placement cell plans for; the times include the greedy comparison computed in the same call.

8.5 Hidden Talent

Over the 12 demo jobs, Hidden Talent finds 10 cases of a consented student within reach of a job they have not applied to, and 9 more cases one step away. For comparison, the same 10 consented students made 14 applications in total. For the job “Backend Engineer – Payments”, for example, it lists two students already within reach (at 100% and 75%) and two who need only Docker. Without the feature a recruiter would see none of them.

8.6 Résumé Skill Finder

On the demo résumé, whose skills section lists Java, Python, JavaScript, SQL, HTML, CSS and Git, the finder reports six further skills found in project and achievement lines (Spring Boot, REST APIs, MySQL, React, Data Structures and Pandas), each with its quoted line. Adding them raises the student’s jobs within reach from 3 to 8, showing that the profile undersold the student.

8.7 Human Evaluation Plan

A human evaluation kit is included with the project. Sixty student–job pairs, stratified by score band, are labelled “good fit” or “not a good fit” by three people who do not see the score. Agreement is measured with Fleiss’ κ [15], and the match rule’s “within reach” decision is compared with a keyword-matching baseline on accuracy, precision, recall and F1, with McNemar’s test for the difference. The labelling is prepared but not yet complete.

8.8 Limitations

- All results are on fictional data; a study over a real placement season is needed to measure the effect on placements.
- The weights (2 and 1), the half credit and the 70% threshold are design choices, not learned from data; unverified levels are self-reported.
- The question bank (8 questions for each of 15 skills) is small enough to memorise over retakes, so it supports screening rather than certification.
- The Workshop Planner assumes every invitee learns the skill and ignores capacity and cost; the Skill Roadmap counts effort in levels, not hours.
- On the free tier of the Gemini API, prompts may be used to improve the provider’s products, so only demo data should be sent there.

CHAPTER 9
CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

CareerSphere AI is a complete campus placement platform for students, recruiters and the placement cell, built around one transparent, level-aware match rule. On top of that rule it answers the practical questions each user has: which skill, or which two or three skills, a student should learn next; which applicants fit a job and why, and which students fit but did not apply; and which workshops the placement cell should run, and for whom. Because skills pay off together, the project treats these as combinatorial questions and answers them by exact search, which is fast at college scale. On the demo batch, a three-step roadmap more than triples what the best single skill achieves for a student (4.71 against 1.43 jobs), and three exactly chosen workshops reach every student where greedy selection leaves three behind.

The project also shows how to use language models responsibly in such a system: the Copilot reads only through tools over the student's own data and the résumé finder keeps only suggestions whose evidence is in the résumé, so the AI never introduces a number or a skill that cannot be checked. The system runs on a laptop with one command, from Xcode, VS Code and Android Studio, and online with Docker, and is covered by 39 unit tests and 17 end-to-end API tests.

9.2 Future Scope

- Complete the human evaluation and run a study over a real placement season.
- Learn the weights and the threshold from past placement outcomes, and calibrate a larger question bank with item difficulty.
- Capacity- and cost-aware workshop planning as an integer program, which also scales beyond exhaustive search.
- Effort measured in hours, with links to specific courses for each roadmap step.
- Email and push notifications for status changes and invitations; interview scheduling with calendars.
- Multi-college support with separate tenants and a shared recruiter directory.
- Native mobile apps with on-device video recording on Android over HTTPS.

REFERENCES

[1] C. de Ruijt and S. Bhulai, “Job recommender systems: A review,” arXiv:2111.13576, 2021.

[2] C. Zhu, H. Zhu, H. Xiong, C. Ma, F. Xie, P. Ding, and P. Li, “Person-job fit: Adapting the right talent for the right job with joint representation learning,” *ACM Trans. Manage. Inf. Syst.*, vol. 9, no. 3, pp. 1–17, 2018.

[3] S. Alfana, S. K. R. Jammalamadaka, V. C. Prakash, and B. T. Reddy, “Cognitive and academic-based probability models for predicting campus placements,” *IAES Int. J. Artif. Intell.*, vol. 11, no. 4, pp. 1239–1251, Dec. 2022, doi: 10.11591/ijai.v11.i4.pp1239-1251.

[4] Y. Zhang and X. Chen, “Explainable recommendation: A survey and new perspectives,” *Found. Trends Inf. Retr.*, vol. 14, no. 1, pp. 1–101, 2020, doi: 10.1561/1500000066.

[5] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, “An analysis of approximations for maximizing submodular set functions—I,” *Math. Program.*, vol. 14, pp. 265–294, 1978.

[6] U. Feige, “A threshold of ln n for approximating set cover,” *J. ACM*, vol. 45, no. 4, pp. 634–652, 1998.

[7] A. Krause and D. Golovin, “Submodular function maximization,” in *Tractability: Practical Approaches to Hard Problems*, L. Bordeaux, Y. Hamadi, and P. Kohli, Eds. Cambridge, U.K.: Cambridge Univ. Press, 2014, pp. 71–104.

[8] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Proc. NeurIPS*, 2023.

[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.

[10] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[11] T. Gao, H. Yen, J. Yu, and D. Chen, “Enabling large language models to generate text with citations,” in *Proc. EMNLP*, 2023, pp. 6465–6488, doi: 10.18653/v1/2023.emnlp-main.398.

[12] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, “Mitigating bias in algorithmic hiring: Evaluating claims and practices,” in *Proc. Conf. Fairness, Accountability, and Transparency (FAT)**, 2020, pp. 469–481.

[13] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.

[14] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proc. USENIX Annu. Tech. Conf., FREENIX Track*, 1999, pp. 81–91.

[15] J. L. Fleiss, “Measuring nominal scale agreement among many raters,” *Psychol. Bull.*, vol. 76, no. 5, pp. 378–382, 1971.

[16] Google, “Function calling with the Gemini API,” Google AI for Developers. [Online]. Available: https://ai.google.dev/gemini-api/docs/function-calling (accessed Oct. 4, 2026).

[17] VMware Tanzu, “Spring Boot Reference Documentation,” version 3.3. [Online]. Available: https://docs.spring.io/spring-boot/

[18] Meta Open Source, “React documentation.” [Online]. Available: https://react.dev/

[19] Oracle, “MySQL 8.0 Reference Manual.” [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[20] OWASP Foundation, “OWASP Top 10: The ten most critical web application security risks,” 2021. [Online]. Available: https://owasp.org/Top10/

APPENDIX A: INSTALLATION AND RUNNING

Prerequisites on a Mac: install Homebrew, then run `brew install openjdk@17 maven node mysql`, link Java with `sudo ln -sfn $(brew --prefix)/opt/openjdk@17/libexec/openjdk.jdk /Library/Java/JavaVirtualMachines/openjdk-17.jdk`, and start the database with `brew services start mysql`.

Start everything with one command:

```
cd careersphere
./run.sh            # starts MySQL, the backend (port 8080) and the web app (
port 5173)
./stop.sh           # stops them
```

Or start the parts by hand:

```
cd backend  && mvn spring-boot:run      # API on http://localhost:8080
cd frontend && npm install && npm run dev  # web app on http://localhost:5173
```

Optional AI key. Put `export GEMINI_API_KEY=...` in `~/.zshrc` (never in the project files). Without it, the Copilot and the résumé finder run in offline mode.

Xcode: open `CareerSphere.xcodeproj` in the project folder, choose the My Mac or an iPhone simulator target and press Run. **VS Code:** open the folder, run the task "CareerSphere: start everything", or use the Run and Debug panel. **Android Studio:** open the `android` folder, start an emulator and press Run; the app connects to `10.0.2.2:5173` by default, which can be changed in its settings.

Docker and online:

```
docker compose up --build     # app and MySQL on http://localhost:8080
# Railway: create a project from the GitHub repository, add a MySQL database,
# set DB_URL, DB_USERNAME, DB_PASSWORD, JWT_SECRET (and optionally GEMINI_API
_KEY)
```

Resetting the demo data: `mysql -u root -e "DROP DATABASE careersphere"` and start again; the seeder recreates the demo batch on an empty database.

Running the tests: `cd backend && mvn test`.

APPENDIX B: DEMO ACCOUNTS AND DEMO SCRIPT

Table 9.1: Demo accounts

Role	Email	Password
Student	student@careersphere.ai	student123
Recruiter	recruiter@careersphere.ai	recruiter123
Placement cell	admin@careersphere.ai	admin123
Other students	e.g. ananya@student.demo	student123

A ten-minute demonstration:

1. Log in as the student. The Career Map shows 3 of 12 jobs within reach. Toggle the Spring Boot lever and watch four jobs move inside the ring.
2. Open Opportunity Math: Spring Boot is first with +4 jobs. Open Skill Roadmap: three steps give +6 jobs.
3. Open Placement profile, upload the sample résumé, accept the six suggested skills, and give consent: jobs within reach rise from 3 to 8.
4. Take a skill assessment and show that the answers are graded on the server.
5. Ask the Copilot “Which skill should I learn next?” and point out the data it used.
6. Apply to a job. Log in as the recruiter, open the job’s applicants, change the status, and open Hidden Talent on “Backend Engineer – Payments” to invite a student.
7. Log in as that student (ananya@student.demo) and show the invitation on the Jobs page.
8. Log in as the placement cell. Verify a pending certificate. Open the Workshop Planner with three workshops: all 35 students reached, against 32 one at a time.

APPENDIX C: COMPLETE LIST OF API ENDPOINTS

Table 9.2: All REST endpoints

Prefix	Endpoints
/api (public)	GET /health; POST /auth/register; POST /auth/login; GET /auth/me; GET /skills
/api/student	GET /dashboard; GET /opportunities; GET /what-if; GET, PUT /profile; POST /profile/skills; POST, GET, DELETE /resume; GET /resume/file; POST /resume/skills; GET /jobs; GET /jobs/{id}; POST, GET /applications; GET /roadmap; GET /invitations; GET /readiness; PUT /consent; GET /leaderboard; POST, GET, DELETE /video; GET /video/file; GET /assessments; POST /assessments/start; POST /assessments/{id}/submit; POST, GET /certificates; DELETE /certificates/{id}; GET /certificates/{id}/file
/api/student/copilot	GET /status; POST (ask)
/api/recruiter	GET /dashboard; GET, PUT /profile; GET, POST /jobs; PUT, DELETE /jobs/{id}; PATCH /jobs/{id}/status; GET /jobs/{id}/applicants; PATCH /applications/{id}/status; GET /applications/{id}/resume; GET /applications/{id}/video; GET /jobs/{id}/talent; POST /jobs/{id}/invite/{studentId}
/api/admin	GET /stats; GET /users; GET /workshops; GET /certificates; PATCH /certificates/{id}; GET /certificates/{id}/file; GET /leaderboard